AKTU OOSD (KCS054 / BCS054) — Solved Question Papers 2020–2025

Oct 24, 2026

How to use these solutions

Every question from all four papers is answered below, year by year, in the order it appears on the paper. Section A answers are sized for 2 marks (definition + one example or difference); Section B and C answers are sized for 10 marks (7 marks in 2024-25) with definition, explanation, diagram or table, program, and a closing point.

All C++ programs compile with any standard C++11 compiler (g++ recommended). Every diagram is drawn in standard UML notation (rounded rectangles for states, underlined names for objects, solid arrows for messages); reproduce them neatly in the exam.

Paper 1: 2020-21 (KCS054, 100 marks)

Section A (2 × 10 = 20)

Q1(a) Define object identity with example. Object identity is the property by which every object is distinct from every other object, even when all their attribute values are equal. Identity is independent of state; in C++ it is usually the object's memory address. Example: two `Student` objects s1 and s2 both with name = "Ravi", roll = 10 are still two different objects (`&s1 != &s2`).

Q1(b) List the features of the object-oriented paradigm. Classes and objects; encapsulation; data abstraction; inheritance; polymorphism; dynamic binding; message passing; reusability and modularity. Emphasis is on data rather than procedure, and programs are built bottom-up.

Q1(c) Define and differentiate link and association. A link is a physical or conceptual connection between two object instances (e.g., *Ravi works-for Infosys*). An association is a group of links with common structure and meaning, defined between classes (e.g., *Person works-for Company*).

Link	Association
Between objects	Between classes
Instance of an association	Template / description of links
Drawn between object boxes with underlined names	Drawn between class boxes, with name and multiplicity

Q1(d) Define and differentiate generalization and inheritance. Generalization is the modelling relationship between a superclass and its subclasses (an *is-a* relationship drawn with a hollow triangle in UML). Inheritance is the mechanism by which a subclass actually acquires the attributes and operations of its superclass. Generalization is the concept seen at design

level; inheritance is how it is implemented in code. Example: `Vehicle` generalizes `Car` and `Bike`; in C++ `class Car : public Vehicle` inherits `speed` and `move()`.

Q1(e) How is multiple inheritance different, and how is it modelled using nested generalization? In multiple inheritance a class has more than one superclass and inherits features from all of them (e.g., `AmphibiousVehicle` from `LandVehicle` and `WaterVehicle`). Nested generalization avoids it by building a single hierarchy: first generalize on one discriminator, then subdivide each subclass on the second (Vehicle → LandVehicle / WaterVehicle → each split by power type). Nested generalization is simpler and avoids ambiguity, but may duplicate classes and lose the "belongs to both" meaning.

Q1(f) Define and differentiate procedural programming and OOP.

Procedural (e.g., C)	Object-oriented (e.g., C++)
Program divided into functions	Program divided into objects
Focus on procedures / algorithms	Focus on data
Data moves freely between functions (global data)	Data hidden, accessed through member functions
Top-down design	Bottom-up design
No inheritance, weak reuse	Inheritance, polymorphism, high reuse

Example: `float area(float r)` in C versus `class Circle { float r; public: float area(); };` in C++.

Q1(g) Program demonstrating all exception-handling keywords (try, catch, throw).

```
#include <iostream>
using namespace std;
int main() {
    int a = 10, b = 0;
    try {
        if (b == 0) throw "Division by zero";   // throw
        cout << a / b;
    }
    catch (const char* msg) {                     // catch
        cout << "Exception: " << msg << endl;
    }
    return 0;
}
```

Q1(h) Define friend function with example. A friend function is a non-member function declared with the keyword `friend` inside a class; it can access the private and protected members of that class.

```
class Box { int w; public: Box(int x){w=x;} friend int getW(Box b); };
int getW(Box b) { return b.w; }   // accesses private w
```

Q1(i) Define and differentiate virtual and pure virtual functions. A virtual function is a base-class member declared `virtual` that can be overridden and is resolved at run time. A pure virtual function has no body in the base class (`= 0`) and makes the class abstract.

Virtual	Pure virtual
Has a definition in base class	No definition (`= 0`)
Overriding optional	Derived class must override
Base class can be instantiated	Base class becomes abstract

Example: `virtual void show(){}` versus `virtual void draw() = 0;`

Q1(j) Define constructor and destructor with example. A constructor is a special member function with the same name as the class, no return type, called automatically when an object is created, to initialise it. A destructor has the class name prefixed with ~, takes no arguments, and is called automatically when the object goes out of scope, to release resources.

```
class Demo { public: Demo(){ cout<<"Created"; } ~Demo()
{ cout<<"Destroyed"; } };
```

Section B (any 3 × 10 = 30)

Q2(a) What is object-oriented technology? Pros and cons with example.

Object-oriented technology (OOT) is a way of building software as a collection of discrete objects that combine data (attributes) and behaviour (operations). It covers the whole life cycle: object-oriented analysis (OOA, identify objects in the problem domain), object-oriented design (OOD, define classes, relationships and interfaces), and object-oriented programming (OOP, implement in a language such as C++ or Java). Its key concepts are identity, classification, inheritance, polymorphism, encapsulation and abstraction.

Example: a banking system models `Account`, `Customer`, `Transaction` as classes. `SavingsAccount` and `CurrentAccount` inherit from `Account` and override `withdraw()`.

Pros:

- Reusability through inheritance and class libraries.
- Modularity: each class can be built and tested separately.
- Easier maintenance and extension; changes stay local because of encapsulation.
- Data security through information hiding.
- Models the real world closely, so analysis maps directly to design and code.
- Polymorphism reduces conditional code and supports plug-in designs.

Cons:

- Steeper learning curve; needs good design skills.
- Programs can be larger and slower (dynamic binding, object overhead).
- Over-design: deep hierarchies become hard to understand.
- Not ideal for every problem (e.g., simple scripts, purely numeric routines).

Q2(b) What is data abstraction? How is it different from encapsulation?

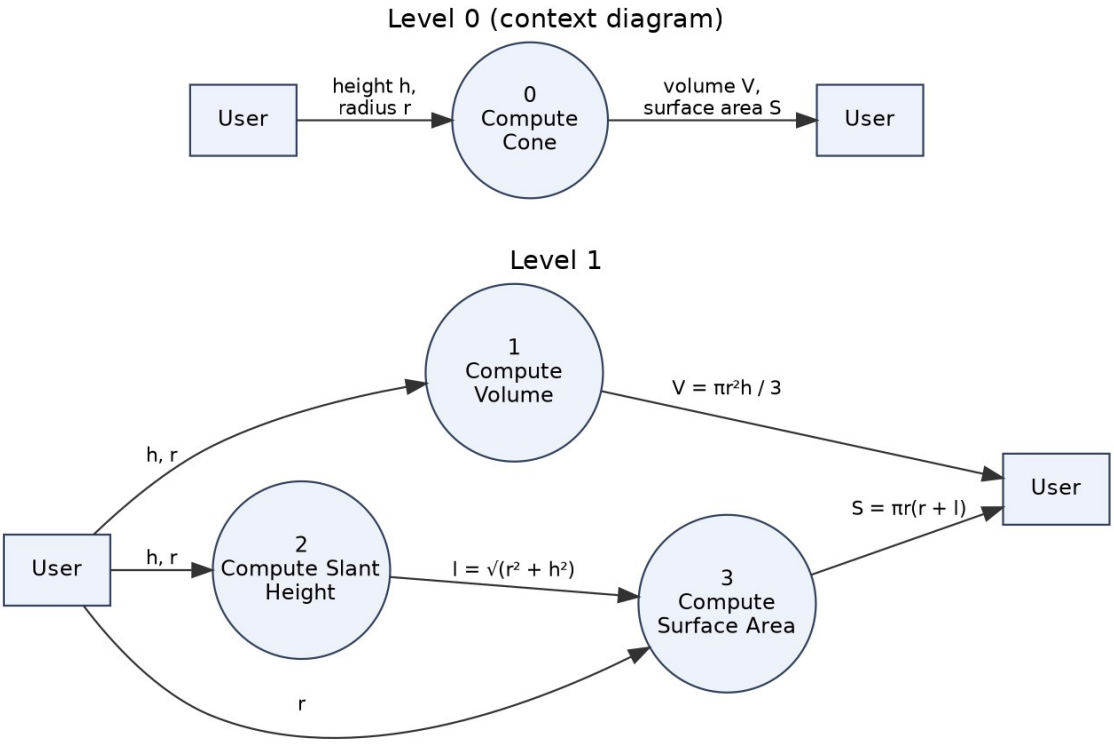
Data abstraction means showing only essential features of an entity and hiding implementation details. The user knows *what* an object does, not *how*. In C++ it is achieved with classes, access specifiers and abstract classes. Encapsulation is wrapping data and the functions that operate on it into one unit (a class) and restricting direct access to the data.

Abstraction	Encapsulation
Hides complexity, shows essentials	Hides data, binds data + code
Design-level concept (what)	Implementation-level concept (how)
Achieved by abstract classes, interfaces	Achieved by classes + private/protected
Focus on external view	Focus on internal protection

```cpp
#include <iostream>
using namespace std;
class Shape {                  // abstraction: only interface
public:
    virtual double area() = 0;
};
class Circle : public Shape {  // encapsulation: r is hidden
    double r;
public:
    Circle(double x) { r = x; }
    double area() { return 3.14 * r * r; }
};
int main() {
    Shape* s = new Circle(2);
    cout << "Area = " << s->area();   // user doesn't know formula
    delete s;
}
```

Q2(c) DFD for computing volume and surface area of a cone; ways of specifying operations.

Formulas: slant height $l = \sqrt{(r^2 + h^2)}$; Volume $V = (1/3)\pi r^2 h$; Surface area $S = \pi r(r + l)$.



Level 0 and Level 1 DFD for volume and surface area of a cone

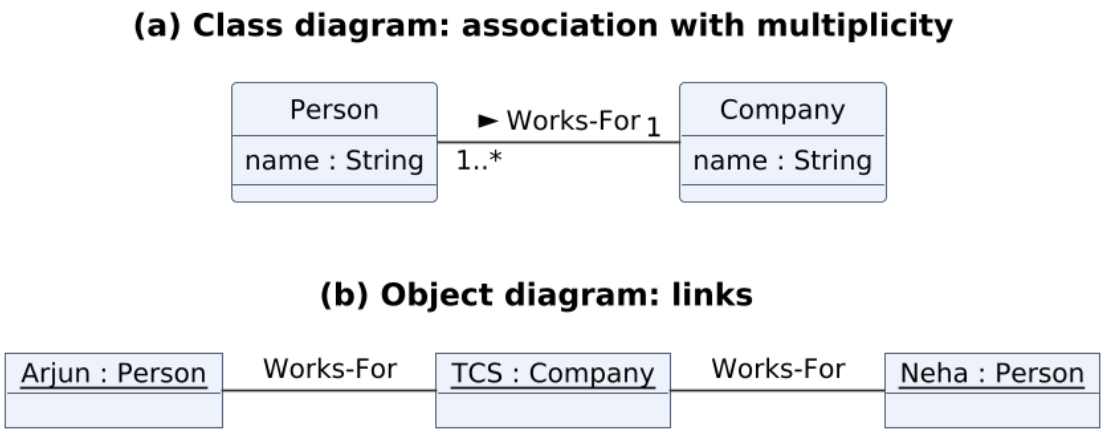
Processes are drawn as circles/ellipses, data flows as labelled arrows, and the external entity (User) as a rectangle.

Ways of specifying operations (functional model):

1. Mathematical functions / formulas (as above).
2. Table of input-output values.
3. Equations specifying output in terms of input.
4. Pre-conditions and post-conditions.
5. Decision tables.
6. Pseudocode or structured English.
7. Natural language description.

Q2(d) Link and association with example; multiplicity.

A link is a connection between two objects (e.g., *Arjun* Works-For *TCS*). An association describes a group of links with common structure and semantics between classes (Person Works-For Company). Associations are bidirectional by default and can be binary, ternary or higher. They may have role names, link attributes and association classes.



Association in a class diagram and links in an object diagram

Multiplicity specifies how many instances of one class may relate to a single instance of the associated class. Notation: `1` (exactly one), `0..1` (optional), * or `0..*` (many), `1..*` (one or more), `2..4` (range). In the example, one company employs 1..* persons, and each person works for exactly 1 company.

Q2(e) Early binding and late binding; techniques used in polymorphism.

Binding is linking a function call to its function body. Early (static, compile-time) binding resolves the call at compile time; it is used for normal functions, function overloading and operator overloading. It is fast. Late (dynamic, run-time) binding resolves the call at run time based on the actual object type; it is implemented with virtual functions accessed through base-class pointers or references (via the vtable). It is flexible.

```
#include <iostream>
using namespace std;
class Base {
public:
    void show()          { cout << "Base show\n"; }   // early binding
    virtual void disp()  { cout << "Base disp\n"; }   // late binding
};
class Derived : public Base {
public:
    void show()  { cout << "Derived show\n"; }
    void disp()  { cout << "Derived disp\n"; }
};
int main() {
    Base* p = new Derived;
    p->show();   // Base show    (decided at compile time)
    p->disp();   // Derived disp (decided at run time)
    delete p;
}
```

Early binding	Late binding
Compile time	Run time

Early binding	Late binding
Overloading (function/operator)	Virtual functions
Faster	Slightly slower (vtable lookup)
Compile-time polymorphism	Run-time polymorphism

Section C (one part from each question, 5 × 10 = 50)

Q3(a) Static member function: characteristics and a justified example.

A static member function is declared with the keyword `static`. It belongs to the class, not to any object, so it can be called as `ClassName::function()` even when no object exists.

Characteristics:

- Can access only static data members and other static functions.
- Has no `this` pointer.
- Cannot be `virtual`, `const` or `volatile`.
- Called with the class name and scope resolution operator (can also be called through an object).
- Only one copy exists for the whole class.

Justified use: counting how many objects have been created. The count belongs to the class as a whole, so both the counter and the function reading it should be static.

```
#include <iostream>
using namespace std;
class Student {
    static int count;        // shared by all objects
    int roll;
public:
    Student(int r) { roll = r; count++; }
    static void showCount() { cout << "Objects: " << count << endl; }
};
int Student::count = 0;      // definition outside class
int main() {
    Student::showCount();    // 0, no object needed
    Student a(1), b(2), c(3);
    Student::showCount();    // 3
}
```

Q3(b) Why is OOP important for the software industry and real life? Pros and cons.

OOP mirrors how we think about the real world: as things (objects) with properties and behaviour. A car has attributes (colour, speed) and behaviour (start, brake); an OOP program models it the same way. This makes large systems easier to understand, divide among teams, and change.

Importance in industry:

- Reuse: frameworks and libraries (STL, Qt, Java collections) are class-based.
- Team development: classes with clear interfaces let teams work in parallel.
- Maintenance: encapsulation localises changes; bugs are easier to trace.
- Extensibility: new features are added by inheriting and overriding, not rewriting.
- Modelling: UML designs map directly to classes.

Example: in an e-commerce system, `Payment` is a base class; `CardPayment`, `UPIPayment`, `NetBanking` override `pay()`. Adding a new wallet needs one new class and no change to checkout code.

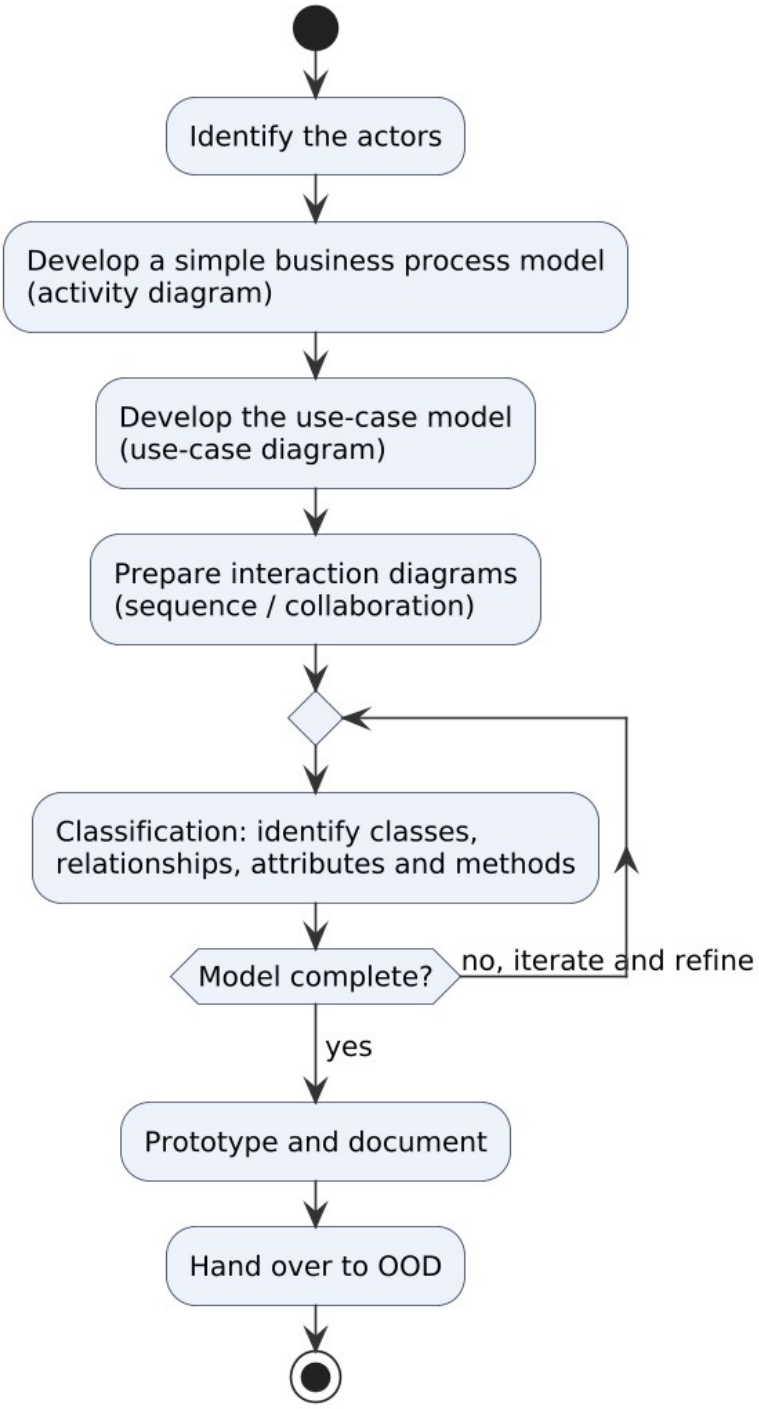
Pros and cons: same as Q2(a) above (reuse, modularity, security, maintainability versus overhead, learning curve, over-design).

Q4(a) What is use-case driven OOA? How is it different from OOD? Explain the OOA process in the unified approach.

Use-case driven OOA means the analysis is guided by use cases: scenarios describing how actors (users or external systems) interact with the system to achieve a goal. Every class, attribute and behaviour found is traced back to some use case, so the model captures what users actually need.

OOA	OOD
What the system must do	How the system will do it
Problem domain objects	Solution / implementation objects
Use cases, domain class model	Detailed class design, interfaces, algorithms
Technology independent	Considers language, DB, UI, architecture

OOA process in the Unified Approach (Bahrami):



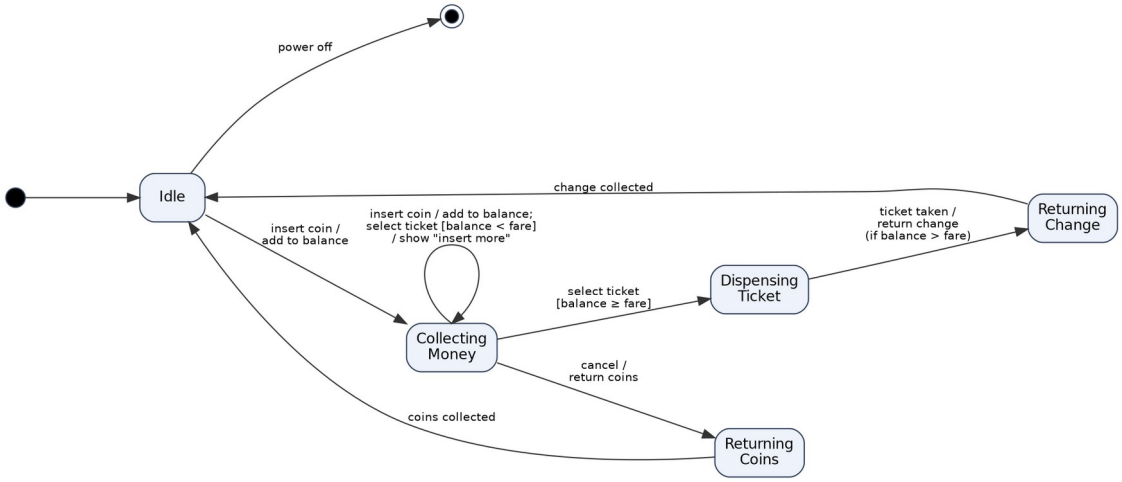
OOA process in the Unified Approach

Steps: (1) identify the actors; (2) develop a business process model; (3) develop the use-case model; (4) prepare interaction diagrams; (5) classify objects using noun-phrase, common-class-pattern, use-case-driven or CRC approaches; identify relationships, attributes and methods; (6) iterate and refine; (7) build prototypes and document.

Q4(b) Properties of a state chart diagram; state chart for a coin vending machine at a railway station.

Properties:

- Shows the states an object passes through during its life, and the events that cause transitions.
- States drawn as rounded rectangles; initial state as a filled circle; final state as a bull's-eye.
- Transitions labelled `event [guard] / action`.
- States can have entry/exit/do activities.
- Supports nested (composite) states and concurrent sub-states.
- Models the dynamic behaviour of one class.



State chart of a coin vending machine at a railway station

Q5(a) Differentiate (i) SA/SD and OMT, (ii) SA/SD and JSD.

(i) SA/SD versus OMT

SA/SD	OMT
Function-oriented: system decomposed into processes	Object-oriented: system decomposed into objects
Functional model (DFD) is primary	Object model is primary, then dynamic and functional
Data dictionary, structure charts, state machines	Class diagrams, state diagrams, DFDs
Changes in requirements affect many functions	Changes stay local to classes
System boundary must be fixed early	Boundary easier to extend
Less reusable	Highly reusable

(ii) SA/SD versus JSD (Jackson Structured Development)

SA/SD	JSD
Starts with functions / data flow	Starts with real-world entities and their actions over time
Uses DFDs and structure charts	Uses entity structure diagrams and network diagrams
Top-down functional decomposition	Models entities as sequential processes
Good for general information systems	Good for real-time / time-ordered systems
Easier to learn, widely used	More complex, less popular

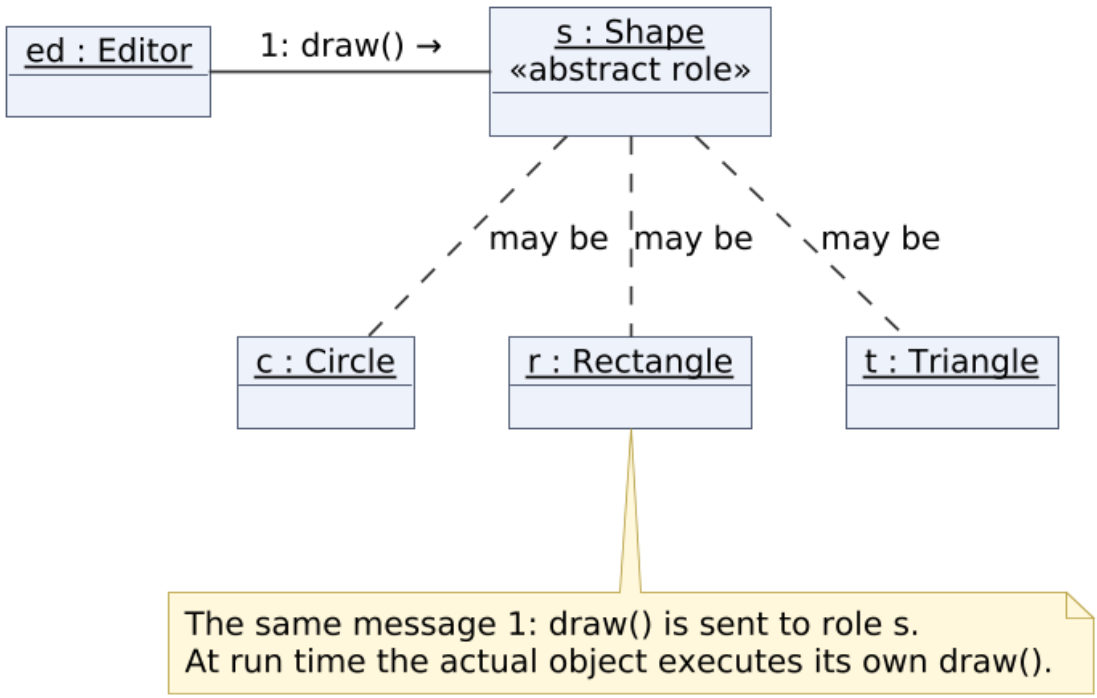
Q5(b) Collaboration diagram: terms, symbols, and polymorphism.

A collaboration (communication) diagram is a UML interaction diagram that shows objects, the links between them, and the numbered messages they exchange. Where a sequence diagram stresses time order, a collaboration diagram stresses structural organisation.

Terms and symbols:

- Object: rectangle with underlined `objectName : ClassName`.
- Link: solid line connecting two objects that communicate.
- Message: small arrow beside a link, with sequence number and name, e.g. `1: placeOrder()`, `1.1: checkStock()` (nested numbering shows call nesting).
- Actor: stick figure that starts the interaction.
- Guard / iteration: `[condition]` and * (e.g., `2*: addItem()`).
- Return values: `3: total := calcTotal()`.

Polymorphism in a collaboration diagram: a message is sent to an object typed by the superclass; the receiving object can be any subclass, and each responds with its own method. It is shown by naming the receiver with the abstract class (or role) and, if needed, a note listing the possible subclasses.



Collaboration diagram showing polymorphism

Example: Editor sends `1: draw()` to `s:Shape`; whether a circle, rectangle or triangle is drawn depends on the actual object, which is polymorphism.

Q6(a) Operator overloading; binary operator overloading with and without friend function.

Operator overloading gives an existing C++ operator an additional meaning for user-defined types, using a function named `operator<symbol>`. Operators that cannot be overloaded: `::`, `.`, `.*`, `?:`, `sizeof`.

Member function (without friend)	Friend function
Takes 1 argument for binary operator (left operand is `*this`)	Takes 2 arguments explicitly
Left operand must be an object of the class	Left operand can be a basic type (e.g., `5 + obj`)
Has `this` pointer	No `this` pointer
Called as `a.operator+(b)`	Called as `operator+(a, b)`
=, `()`, `[]`, `->` can be overloaded only this way	Cannot overload =, `()`, `[]`, `->`

```
#include <iostream>
using namespace std;
class Complex {
    float re, im;
public:
    Complex(float r = 0, float i = 0) { re = r; im = i; }
```

```
    // 1) member function: one argument
    Complex operator+(Complex c) { return Complex(re + c.re, im +
c.im); }
    // 2) friend function: two arguments
    friend Complex operator-(Complex a, Complex b);
    void show() { cout << re << " + " << im << "i\n"; }
};
Complex operator-(Complex a, Complex b) { return Complex(a.re - b.re,
a.im - b.im); }
int main() {
    Complex a(5, 4), b(2, 1);
    (a + b).show();   // 7 + 5i   (member)
    (a - b).show();   // 3 + 3i   (friend)
}
```

Q6(b) Overloading versus overriding; program to overload + using friend function to concatenate two strings.

Overloading	Overriding
Same name, different parameter list	Same name and same signature
Within same class (or scope)	Between base and derived class
Compile-time polymorphism	Run-time polymorphism (with `virtual`)
Inheritance not required	Inheritance required
Return type may differ	Return type must be same (or covariant)

```
#include <iostream>
#include <cstring>
using namespace std;
class String {
    char s[100];
public:
    String(const char* str = "") { strcpy(s, str); }
    friend String operator+(String a, String b);
    void show() { cout << s << endl; }
};
String operator+(String a, String b) {
    String t;
    strcpy(t.s, a.s);
    strcat(t.s, b.s);
    return t;
}
int main() {
    String s1("Object "), s2("Oriented");
    String s3 = s1 + s2;
    s3.show();          // Object Oriented
}
```

Q7(a) Inheritance in C++; containership and how it differs; avoiding ambiguity in single inheritance using scope resolution.

Inheritance is the process by which a new (derived) class acquires the properties of an existing (base) class: `class D : public B`. Types: single, multiple, multilevel, hierarchical, hybrid. It provides reuse and an *is-a* relationship.

Containership (composition) means a class contains an object of another class as a data member, giving a *has-a* relationship. Example: `class Car { Engine e; };`.

Inheritance	Containership
is-a relationship	has-a relationship
Derived class gets base members directly	Container uses contained object's public interface
Supports overriding / polymorphism	No overriding
Tighter coupling	Looser coupling

Ambiguity in single inheritance: when base and derived classes have a function with the same name, the derived version hides the base version. The scope resolution operator `::` selects the intended one.

```
#include <iostream>
using namespace std;
class A {
public:
    void display() { cout << "Display of A\n"; }
};
class B : public A {
public:
    void display() { cout << "Display of B\n"; }
    void both() { A::display(); display(); }
};
int main() {
    B obj;
    obj.display();      // Display of B
    obj.A::display();   // Display of A  (ambiguity resolved)
    obj.both();
}
```

Q7(b) Pointers in C++, const pointers, uses of pointers, size of void pointer.

A pointer is a variable that stores the memory address of another variable: `int x = 10; int* p = &x;` where `*p` gives 10.

Const pointers (three forms):

- Pointer to constant: `const int* p;` — the value cannot be changed through p; p can point elsewhere.

- Constant pointer: `int* const p = &x;` — p cannot point elsewhere; value can change.
- Constant pointer to constant: `const int* const p = &x;` — neither can change.

Uses: dynamic memory allocation (`new`/`delete`); passing large objects efficiently and call by reference; arrays and strings; data structures (linked lists, trees); run-time polymorphism through base-class pointers; function pointers / callbacks; `this` pointer.

Void pointer: a generic pointer `void*` that can hold the address of any type but must be cast before dereferencing. Its size equals the size of any data pointer on the platform: 4 bytes on a 32-bit system, 8 bytes on a 64-bit system.

```
#include <stdio.h>
int main() {
    int a = 5; float b = 2.5f;
    void* vp;
    vp = &a; printf("%d\n", *(int*)vp);     // 5
    vp = &b; printf("%.1f\n", *(float*)vp); // 2.5
    printf("Size of void pointer = %zu\n", sizeof(vp)); // 8 on 64-
bit, 4 on 32-bit
    return 0;
}
```

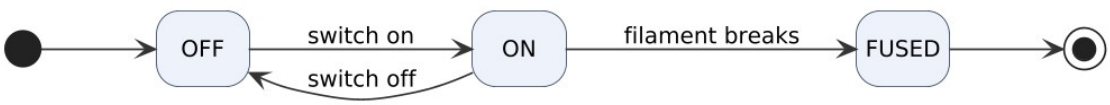
Paper 2: 2021-22 (KCS054, 100 marks)

Section A (2 × 10 = 20)

Q1(a) Differentiate between link and association. A link connects two objects (instance level, e.g., *Ravi* works-for *Wipro*); an association connects two classes and describes a set of links with common meaning (Employee works-for Company). A link is an instance of an association. Links appear in object diagrams; associations in class diagrams with names and multiplicity.

Q1(b) What is an abstract class? Can it be inherited? An abstract class is a class that cannot be instantiated because it contains at least one pure virtual function (`virtual void f() = 0;`). Yes: an abstract class is meant to be inherited. The derived class must override all pure virtual functions to become concrete; otherwise it also remains abstract.

Q1(c) Draw a state diagram for an electric bulb.



State diagram of an electric bulb

Q1(d) When do we use the protected visibility specifier? When a member must be hidden from the outside world (like private) but must be directly accessible to derived classes. It is used for data that subclasses need during inheritance, e.g., `protected: float balance;` in `Account` used by `SavingsAccount`.

Q1(e) When will you make a function inline? Why? When the function is small (one or two lines), called frequently, and contains no loops, recursion, switch, goto or static variables. Inline expansion replaces the call with the function body, removing call overhead (stack push/pop, jump) and so speeding up execution.

Q1(f) Define destructor. Why is it used? A destructor is a member function with the class name preceded by ~, no arguments and no return type, invoked automatically when an object is destroyed. It releases resources acquired by the object (heap memory, files, connections), preventing memory leaks. Example: `~Stack() { delete[] arr; }`.

Q1(g) Use of the scope resolution operator (::).

1. Access a global variable hidden by a local variable: `::x`.
2. Define a member function outside the class: `void A::show() {}`.
3. Access static members: `A::count`.
4. Resolve ambiguity in inheritance: `obj.Base::show()`.
5. Access namespace members: `std::cout`.

Q1(h) What is a candidate key? A candidate key is a minimal set of attributes that uniquely identifies each object (or tuple) in a class or association; no attribute can be removed without losing uniqueness. In OMT it is used to identify links of an association. Example: in `Student(roll, email, name)`, both `roll` and `email` are candidate keys; one is chosen as the primary key.

Q1(i) Use of friend function in C++. A friend function can access private and protected members of a class without being a member. It is used to operate on objects of two different classes (e.g., compare two classes' private data), to overload operators where the left operand is not an object (`5 + obj`, `cout << obj`), and to provide controlled access to helper functions.

Q1(j) What is information hiding? Information hiding is the principle of hiding the internal details (data and implementation) of an object from other objects, exposing only a public interface. In C++ it is achieved by declaring data members `private/protected` and accessing them through public methods. It protects data from accidental change and allows the implementation to change without affecting users.

Section B (any 3 × 10 = 30)

Q2(a) Constructor: characteristics and program for constructor overloading.

A constructor is a special member function that initialises an object when it is created. It has the same name as the class and is called automatically.

Characteristics:

- Same name as the class; no return type, not even `void`.
- Invoked automatically at object creation.
- Should be declared `public`.
- Can have default arguments and can be overloaded.

- Cannot be `virtual`, `static` or `const`; cannot be inherited (though the base constructor is called by the derived one).
- Its address cannot be taken.
- Types: default, parameterised, copy, dynamic constructors.
- If none is written, the compiler supplies a default constructor.

```
#include <iostream>
using namespace std;
class Rectangle {
    int len, wid;
public:
    Rectangle()               { len = wid = 0; }          // default
    Rectangle(int s)          { len = wid = s; }          // one
parameter
    Rectangle(int l, int w)   { len = l; wid = w; }       // two
parameters
    Rectangle(const Rectangle& r) { len = r.len; wid = r.wid; } //
copy
    int area() { return len * wid; }
};
int main() {
    Rectangle r1, r2(5), r3(4, 6), r4(r3);
    cout << r1.area() << " " << r2.area() << " "
         << r3.area() << " " << r4.area() << endl;   // 0 25 24 24
}
```

Q2(b) Inline function: why it is used, with program.

An inline function is a function whose code is substituted at the point of call by the compiler instead of performing a normal call. It is declared with the keyword `inline` (functions defined inside a class are inline automatically). `inline` is a request; the compiler may ignore it for functions with loops, recursion, static variables, switch or goto.

Why use it: removes function-call overhead (saving registers, pushing arguments, jumping, returning); faster for small, frequently called functions; type-safe unlike macros.

```
#include <iostream>
using namespace std;
inline int square(int x) { return x * x; }
inline float cube(float x) { return x * x * x; }
int main() {
    cout << "Square of 5 = " << square(5) << endl;   // 25
    cout << "Cube of 2.5 = " << cube(2.5) << endl;   // 15.625
}
```

Q2(c) OOP, advantages over procedural programming, and a program with class `integer` printing primes.

Object-oriented programming organises a program around objects, which are instances of classes that bind data and functions together. Objects communicate by message passing (calling each other's member functions).

Advantages over procedural programming:

- Data security through encapsulation and data hiding (no global free-flowing data).
- Code reuse through inheritance.
- Polymorphism gives flexible, extensible code.
- Easier to model real-world problems and to maintain large programs.
- Modular: objects can be developed and tested independently.
- Bottom-up design supports parallel team development.

```cpp
#include <iostream>
using namespace std;
class integer {
    int n;                          // print primes from 2 to n
public:
    void read() { cout << "Enter limit: "; cin >> n; }
    bool isPrime(int x) {
        if (x < 2) return false;
        for (int i = 2; i * i <= x; i++)
            if (x % i == 0) return false;
        return true;
    }
    void printPrimes() {
        cout << "Primes up to " << n << ": ";
        for (int i = 2; i <= n; i++)
            if (isPrime(i)) cout << i << " ";
        cout << endl;
    }
};
int main() {
    integer obj;
    obj.read();           // e.g. 20
    obj.printPrimes();    // 2 3 5 7 11 13 17 19
}
```

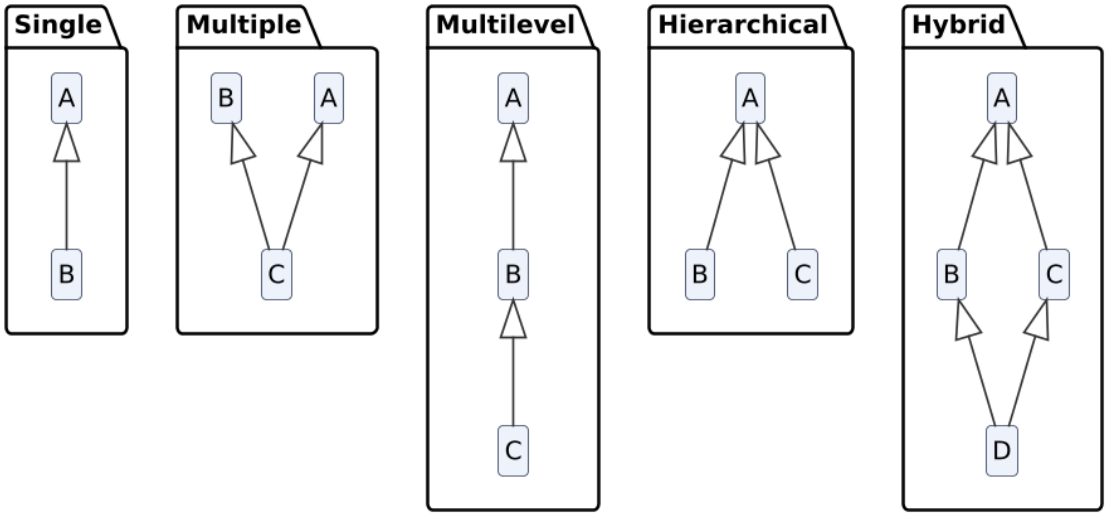
Q2(d) Explain all basic concepts of OOP.

1. Object: a run-time entity with identity, state and behaviour (e.g., `myCar`).
2. Class: a blueprint/user-defined type describing objects with common attributes and operations (e.g., `Car`).
3. Encapsulation: wrapping data and functions into a single unit (class).
4. Data abstraction: representing essential features while hiding background details.
5. Information hiding: data made private and accessed only through public methods.
6. Inheritance: a new class acquires properties of an existing class (is-a; reuse).
7. Polymorphism: one name, many forms — compile time (overloading) and run time (virtual functions).
8. Dynamic binding: the code executed for a call is decided at run time.
9. Message passing: objects communicate by invoking methods: `object.method(args)`.

A short example ties them together: `class Shape { protected: string name; public: virtual double area()=0; };` (abstraction, encapsulation); `class Circle : public Shape` (inheritance); `Shape* s = new Circle; s->area();` (polymorphism, dynamic binding, message passing).

Q2(e) Inheritance and its types; program for multiple inheritance.

Inheritance is the mechanism of deriving a new class (derived/child) from an existing class (base/parent) so that the derived class reuses and extends the base class's members.



Types of inheritance

- Single: one base, one derived.
- Multiple: one derived class from two or more bases.
- Multilevel: derived class becomes base for another class.
- Hierarchical: several classes derived from one base.
- Hybrid: combination of two or more types (needs virtual base classes to avoid duplication).

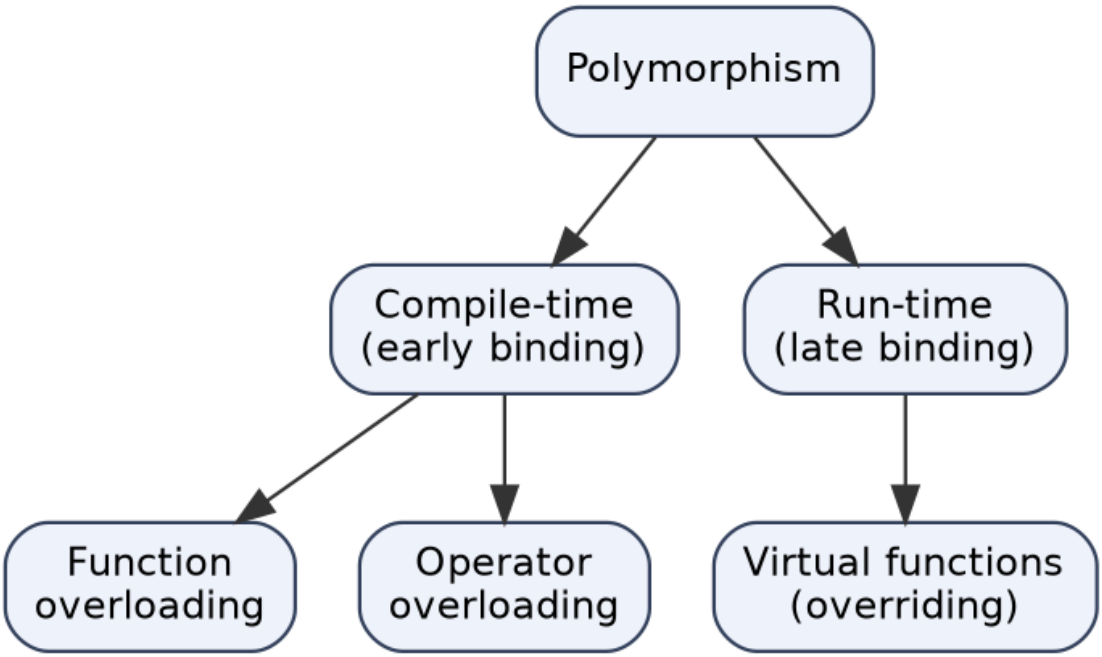
```
#include <iostream>
using namespace std;
class Student {
protected: int roll;
public: void getRoll(int r) { roll = r; }
};
class Sports {
protected: int score;
public: void getScore(int s) { score = s; }
};
class Result : public Student, public Sports {   // multiple
inheritance
public:
    void display() {
        cout << "Roll: " << roll << ", Sports score: " << score <<
endl;
```

```
    }
};
int main() {
    Result r;
    r.getRoll(101);
    r.getScore(85);
    r.display();      // Roll: 101, Sports score: 85
}
```

Section C (one part from each question, 5 × 10 = 50)

Q3(a) Polymorphism; compile-time versus run-time polymorphism.

Polymorphism (Greek: many forms) is the ability of one name or interface to behave differently depending on the type of data or object it acts on.



Classification of polymorphism

Compile-time (static)	Run-time (dynamic)
Call resolved by compiler	Call resolved during execution
Function and operator overloading	Function overriding with virtual functions
Early binding	Late binding (vtable)
Faster	Slower but more flexible
Inheritance not needed	Needs inheritance + base pointer/reference

```
#include <iostream>
using namespace std;
class Calc {                        // compile-time
public:
```

```
    int add(int a, int b)           { return a + b; }
    double add(double a, double b) { return a + b; }
};
class Animal {                      // run-time
public: virtual void sound() { cout << "Animal sound\n"; }
};
class Dog : public Animal {
public: void sound() { cout << "Bark\n"; }
};
int main() {
    Calc c;
    cout << c.add(2, 3) << " " << c.add(2.5, 1.5) << endl; // 5 4
    Animal* a = new Dog;
    a->sound();                     // Bark
    delete a;
}
```

Q3(b) Operator overloading; program for binary operator overloading.

Operator overloading lets C++ operators work with user-defined types by defining a function `return_type operator op (args)`. Rules: only existing operators can be overloaded; at least one operand must be user-defined; precedence, associativity and arity cannot change; `::`, `.`, `.*`, `?:`, `sizeof` cannot be overloaded. A binary operator as a member function takes one argument; as a friend, two.

```
#include <iostream>
using namespace std;
class Distance {
    int feet, inch;
public:
    Distance(int f = 0, int i = 0) { feet = f; inch = i; }
    Distance operator+(Distance d) {          // binary +
        Distance t;
        t.inch = inch + d.inch;
        t.feet = feet + d.feet + t.inch / 12;
        t.inch %= 12;
        return t;
    }
    void show() { cout << feet << "' " << inch << "\"\n"; }
};
int main() {
    Distance d1(5, 9), d2(3, 7);
    Distance d3 = d1 + d2;   // d1.operator+(d2)
    d3.show();               // 9' 4"
}
```

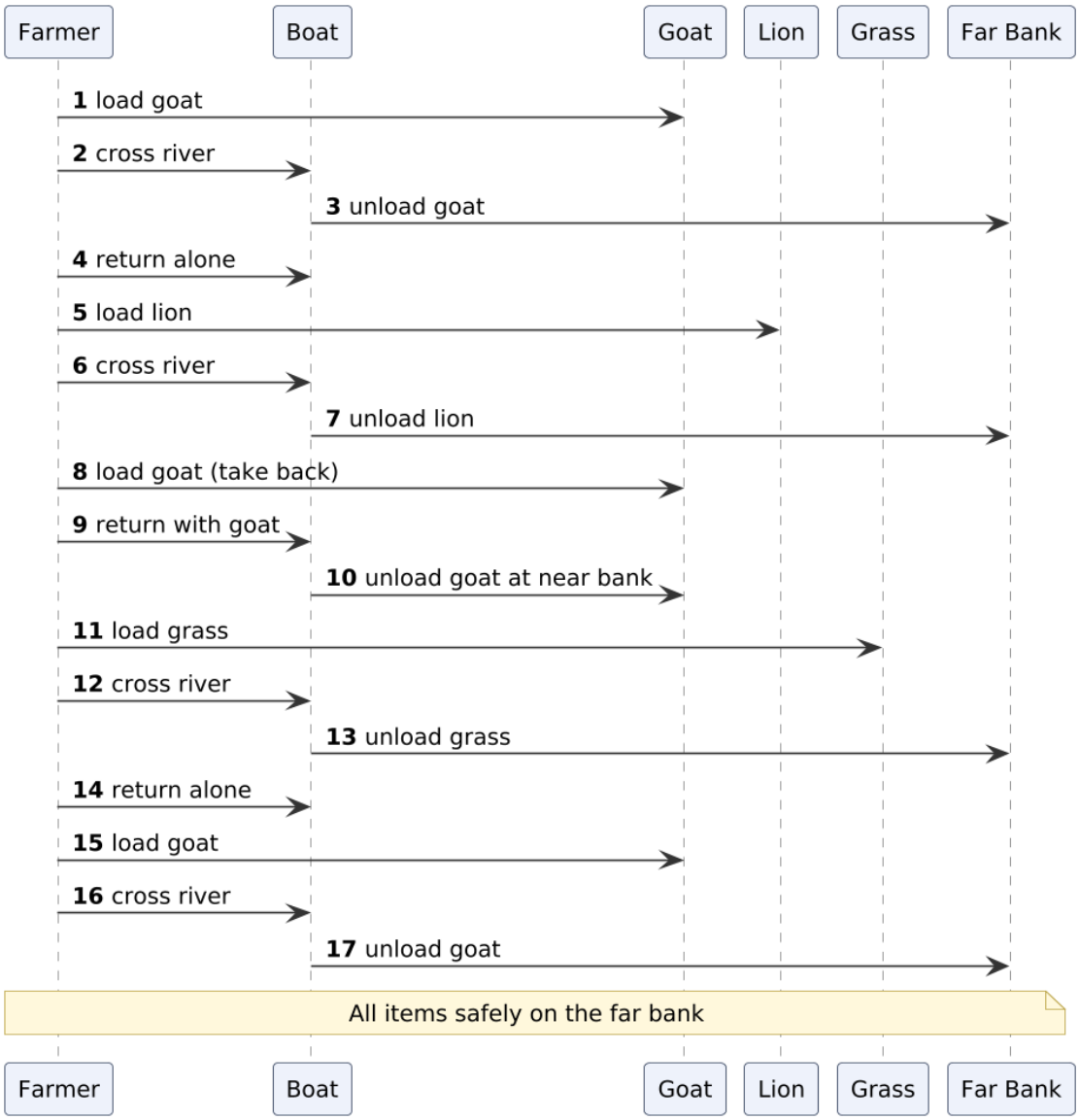
Q4(a) Farmer, goat, lion and grass: (i) scenario, (ii) event trace diagram.

(i) Scenario (the boat carries the farmer and one item; lion does not eat grass):

1. Farmer takes the goat across. (Left: lion, grass — safe.)

2. Farmer returns alone.
3. Farmer takes the lion across. (Right: lion, goat.)
4. Farmer brings the goat back. (Right: lion alone — safe.)
5. Farmer takes the grass across. (Left: goat alone; Right: lion, grass — safe.)
6. Farmer returns alone.
7. Farmer takes the goat across. Everything is safely on the far bank.

(ii) Event trace diagram (objects as vertical lines, events as horizontal arrows in time order):

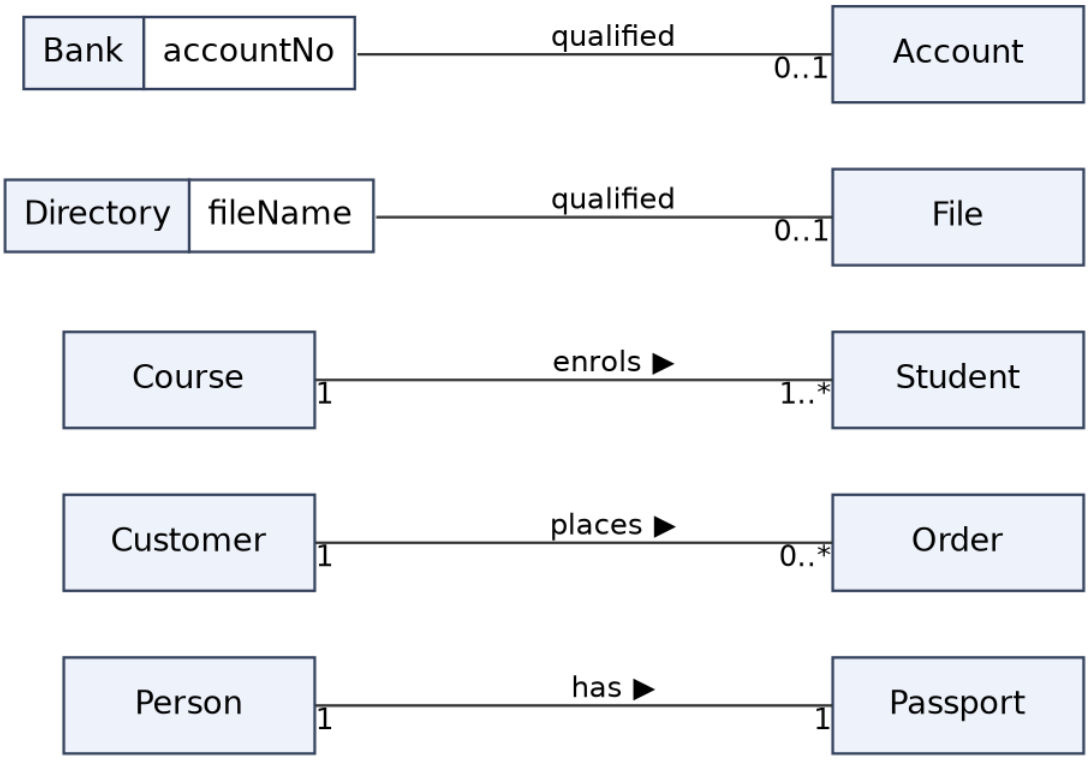


Event trace diagram for the farmer, goat, lion and grass problem

Q4(b) Multiplicity and quantification with examples.

Multiplicity specifies how many instances of one class may be related to a single instance of an associated class. It constrains the number of related objects and is written at the association end.

Notation	Meaning	Example
`1`	exactly one	Each Person has 1 Passport
`0..1`	zero or one	Employee manages 0..1 Department
`*` / `0..*`	zero or more	Customer places * Orders
`1..*`	one or more	Course has 1..* Students
`m..n`	range	Car has 4..5 Wheels



Multiplicity and qualified associations

Quantification (qualification) uses a qualifier, an attribute on the association end, to reduce the multiplicity by selecting specific objects (it partitions the set of related objects). Example: a `Directory` contains many `Files`, but `Directory + fileName` identifies exactly one file. Similarly, `Bank + accountNo` gives exactly one `Account`.

Q5(a) What is UML? Building blocks of UML; types of things.

UML (Unified Modeling Language) is a standard visual language, managed by the OMG, for specifying, visualising, constructing and documenting the artifacts of software systems. It was created by Booch, Rumbaugh and Jacobson.

Building blocks:

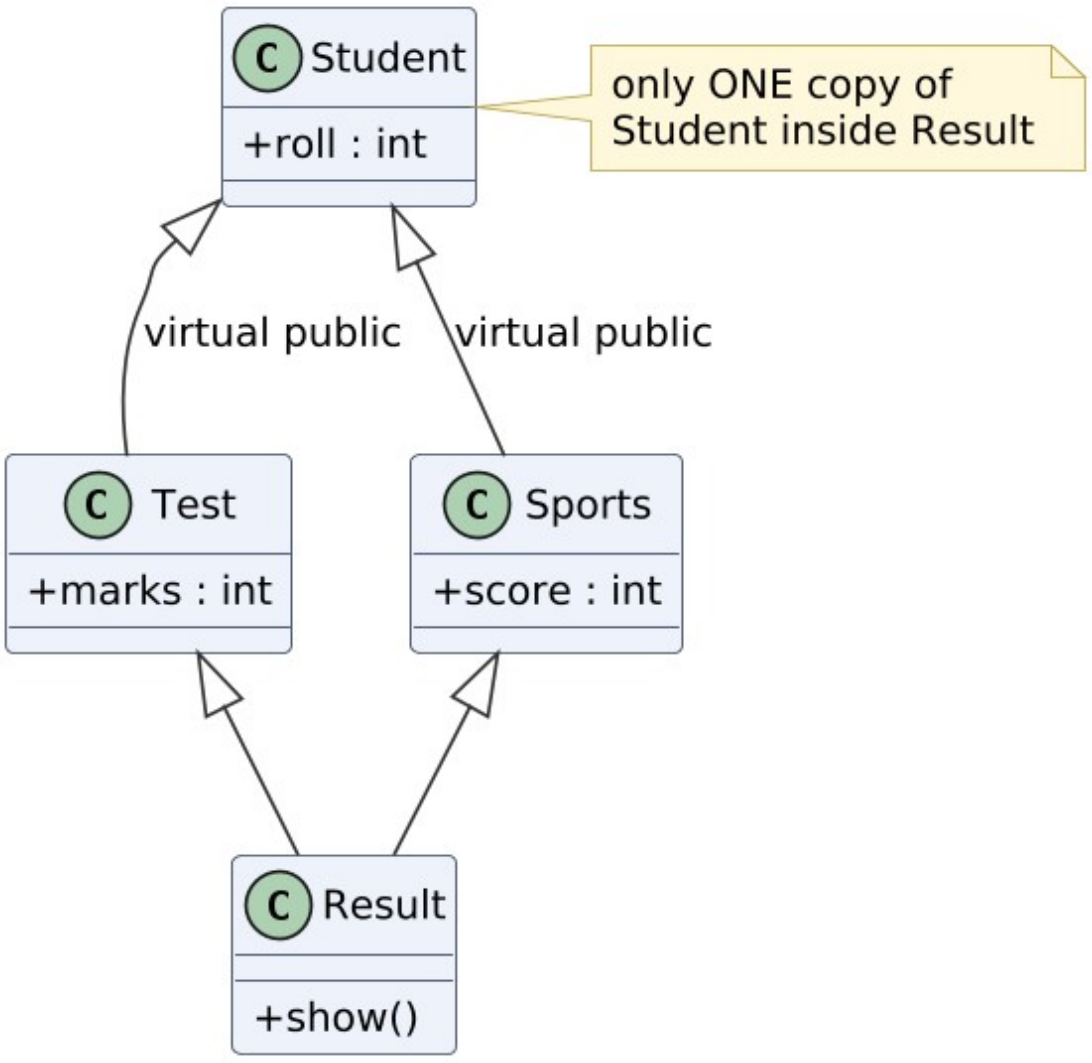
1. Things — the basic elements.
2. Relationships — dependency, association, generalization, realization.
3. Diagrams — structural (class, object, component, deployment, package, composite structure) and behavioural (use case, sequence, collaboration/communication, state chart, activity, timing, interaction overview).

Types of things:

- Structural things (nouns, static parts): class, interface, collaboration, use case, active class, component, node.
- Behavioural things (verbs, dynamic parts): interaction (messages exchanged), state machine (sequence of states).
- Grouping things: package — organises elements into groups.
- Annotational things: note — comments and explanations attached to elements.

Q5(b) Virtual base class with an example.

In hybrid (diamond) inheritance, class D inherits from B and C, which both inherit from A. D then gets two copies of A's members, causing ambiguity. Declaring A as a virtual base class (`class B : virtual public A`) ensures only one shared copy of A is inherited.



Diamond inheritance with virtual base class

```
#include <iostream>
using namespace std;
class Student { public: int roll; };
class Test    : virtual public Student { public: int marks; };
class Sports : virtual public Student { public: int score; };
class Result : public Test, public Sports {
public:
    void show() { cout << "Roll " << roll << " Total " << marks +
score << endl; }
};
int main() {
    Result r;
    r.roll = 7;         // no ambiguity: single copy of Student
    r.marks = 80; r.score = 15;
    r.show();           // Roll 7 Total 95
}
```

Q6(a) Implicit and explicit type conversion.

Type conversion changes a value from one data type to another.

Implicit conversion (automatic / type promotion) is done by the compiler without programmer intervention, when operands of different types are mixed. Lower types are promoted to higher types to avoid data loss: `bool → char → short → int → long → float → double → long double`.

```
int i = 10; float f = 2.5;
double r = i + f;   // i converted to float, result to double = 12.5
char c = 'A'; int x = c + 1;   // 66
```

Risks: loss of sign (signed to unsigned), truncation when assigning a larger type to a smaller one (`int n = 3.9;` gives 3).

Explicit conversion (type casting) is requested by the programmer.

- C-style: `(float)a / b`
- Function-style: `float(a) / b`
- C++ cast operators:
  - `static_cast<T>` — normal compile-time conversions (`static_cast<double>(a)`).
  - `dynamic_cast<T>` — safe downcasting in polymorphic hierarchies, checked at run time.
  - `const_cast<T>` — adds or removes `const`.
  - `reinterpret_cast<T>` — low-level bit reinterpretation between unrelated pointer types.

```
#include <iostream>
using namespace std;
int main() {
    int a = 7, b = 2;
    cout << a / b << endl;                          // 3   (integer
division)
    cout << (float)a / b << endl;                   // 3.5 (C-style)
    cout << static_cast<double>(a) / b << endl;  // 3.5 (C++ cast)
}
```

Conversions involving classes: basic → class (constructor with one argument), class → basic (overloaded casting operator `operator int()`), class → class (constructor or casting operator).

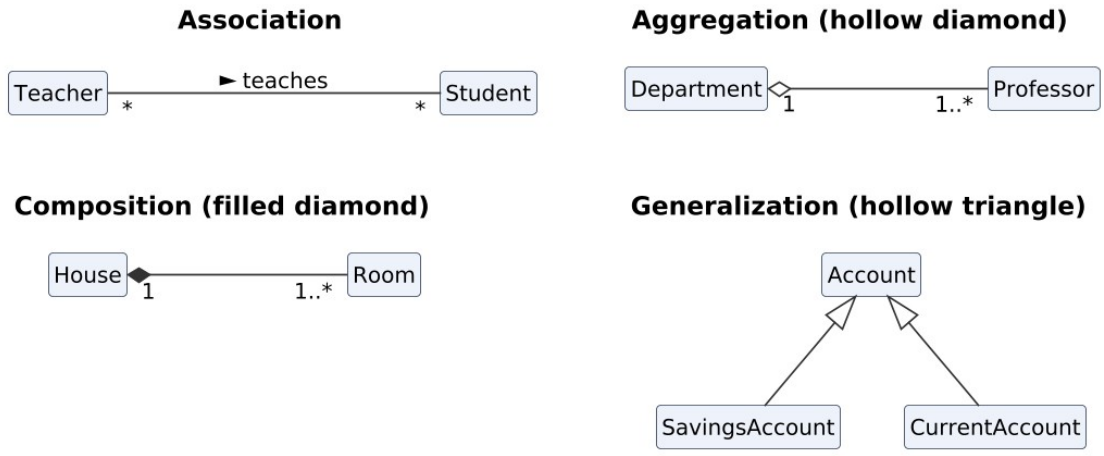
Q6(b) Generalization, aggregation and association in detail.

Association is a structural relationship describing a set of links between objects of classes. It has a name, role names and multiplicity, and can be unary, binary or ternary. Example: `Teacher teaches Student` (many-to-many).

Aggregation is a special form of association representing a whole-part (has-a) relationship where parts can exist independently of the whole. Drawn with a hollow diamond at the whole

end. Example: `Department` has `Professors`; if the department is closed, professors still exist. Properties: transitivity and antisymmetry. A stronger form, composition (filled diamond), means the part's lifetime depends on the whole (`House` – `Room`).

Generalization is the is-a relationship between a general class (superclass) and specialised classes (subclasses). Subclasses inherit attributes, operations and relationships. Drawn with a hollow triangle pointing to the superclass. Example: `Account` generalizes `SavingsAccount` and `CurrentAccount`. A discriminator names the attribute on which the generalization is based.



Association, aggregation, composition and generalization notation

Basis	Association	Aggregation	Generalization
Relationship	uses / knows	has-a (whole-part)	is-a
Level	objects related	objects composed	classes classified
Notation	plain line	hollow diamond	hollow triangle
Inheritance	No	No	Yes

Q7(a) Friend function in C++ with program.

A friend function is a non-member function given access to the private and protected members of a class by declaring it inside the class with the `friend` keyword.

Characteristics:

- Not in the scope of the class; called like a normal function (no object or dot operator).
- Cannot access members directly by name; uses object name and dot (`obj.x`).
- Can be declared in public or private section with the same effect.
- Usually takes objects as arguments.
- Can be a friend of more than one class.
- Friendship is not inherited, not mutual and not transitive.

```cpp
#include <iostream>
using namespace std;
class B;                          // forward declaration
class A {
    int x;
public:
    A(int v) { x = v; }
    friend int maxOf(A, B);
};
class B {
    int y;
public:
    B(int v) { y = v; }
    friend int maxOf(A, B);
};
int maxOf(A a, B b) {             // accesses private data of both
    return (a.x > b.y) ? a.x : b.y;
}
int main() {
    A a(25); B b(40);
    cout << "Maximum = " << maxOf(a, b) << endl;   // 40
}
```

Q7(b) (i) this pointer, (ii) array of objects, with example.

(i) `this` is an implicit pointer available inside every non-static member function; it holds the address of the object that invoked the function. Uses: resolve name clash between data members and parameters (`this->x = x`); return the current object (`return *this`) for chaining; pass the current object to other functions. Static member functions have no `this`.

(ii) An array of objects is an array whose elements are objects of a class: `Employee e[3];`. Each element is accessed with an index and the dot operator: `e[i].getData()`. The default constructor is called for each element.

```cpp
#include <iostream>
using namespace std;
class Employee {
    int id; float salary;
public:
    Employee& set(int id, float salary) {
        this->id = id;              // this resolves name clash
        this->salary = salary;
        return *this;               // returns current object
    }
    void show() { cout << id << "  " << salary << endl; }
};
int main() {
    Employee e[3];                  // array of objects
    e[0].set(1, 30000); e[1].set(2, 42000); e[2].set(3, 51000);
```

```
    for (int i = 0; i < 3; i++) e[i].show();
}
```

Paper 3: 2023-24 (KCS054, 100 marks)

Section A (2 × 10 = 20)

Q1(a) Features of object-oriented languages. Support for classes and objects; encapsulation and data hiding (access specifiers); inheritance; polymorphism (overloading and overriding); dynamic binding; message passing; abstraction through abstract classes/interfaces; constructors/destructors for object lifecycle; exception handling and templates/generics for reuse. Examples: C++, Java, Python, C#.

Q1(b) Structured approach versus object-oriented approach.

Structured	Object-oriented
System seen as functions/processes	System seen as interacting objects
Top-down functional decomposition	Bottom-up, based on classes
Data and functions separate	Data and functions bound in classes
DFD, structure charts	Class, sequence, state diagrams (UML)
Low reuse, harder to change	High reuse, easier maintenance

Q1(c) What is UML? UML (Unified Modeling Language) is a standard, general-purpose graphical language, adopted by the OMG in 1997, for specifying, visualising, constructing and documenting software systems. It was unified from the methods of Booch, Rumbaugh (OMT) and Jacobson (OOSE). It provides structural diagrams (class, object, component, deployment, package) and behavioural diagrams (use case, sequence, collaboration, state chart, activity).

Q1(d) Describe generalization. Generalization is the is-a relationship between a superclass and one or more subclasses, where common attributes and operations are placed in the superclass and inherited by subclasses. It is shown by a hollow triangle pointing to the superclass. Example: `Employee` generalizes `Manager` and `Engineer`. The reverse view is specialization.

Q1(e) Three models in OMT.

1. Object model: static structure — classes, objects, attributes, operations, relationships (class/object diagrams).
2. Dynamic model: behaviour over time — events, states, transitions (state diagrams, event traces).
3. Functional model: data transformations — how output values are computed from inputs (data flow diagrams).

Q1(f) Optimization of design. Design optimization refines the analysis model into an efficient implementation design. It includes: adding redundant associations for efficient access paths; rearranging the execution order of computations; saving derived attributes to avoid recomputation; adjusting inheritance (abstracting common behaviour, rearranging classes);

and choosing efficient algorithms and data structures. The goal is better performance without losing clarity.

Q1(g) C++ program to calculate sin(x). Uses the Taylor series $\sin x = x - x^3/3! + x^5/5! - \ldots$

```
#include <iostream>
using namespace std;
int main() {
    double deg, x, term, sum; int n = 10;
    cout << "Angle in degrees: "; cin >> deg;
    x = deg * 3.14159265 / 180;  term = sum = x;
    for (int i = 1; i <= n; i++) {
        term = -term * x * x / ((2*i) * (2*i + 1));
        sum += term;
    }
    cout << "sin(" << deg << ") = " << sum << endl;   // 30 -> 0.5
}
```

Q1(h) Typecasting in C++. Typecasting is explicitly converting a value from one type to another. Forms: C-style `(int)3.7`, function style `int(3.7)`, and C++ operators `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`. Example: `double avg = static_cast<double>(sum) / n;` avoids integer division.

Q1(i) Public versus private member function.

Public member function	Private member function
Accessible from anywhere through an object	Accessible only inside the class (and friends)
Forms the class interface	Helper / internal utility
Called as `obj.f()` from main	Called only by other member functions

Q1(j) Static data member and static member function. A static data member is shared by all objects of a class; only one copy exists, it is initialised to zero by default, and it must be defined outside the class (`int A::count = 0;`). A static member function can access only static members, has no `this` pointer, and is called as `A::show()`. Typical use: counting objects created.

Section B (any 3 × 10 = 30)

Q2(a) Object-oriented technology: pros and cons with example.

This is the same question as 2020-21 Q2(a); write that full answer. Structure for 10 marks: definition of OOT (software as a collection of objects combining data and behaviour, covering OOA → OOD → OOP); its six concepts (identity, classification, inheritance, polymorphism, encapsulation, abstraction); the banking example (`Account` base class, `SavingsAccount`/`CurrentAccount` overriding `withdraw()`); then the pros list (reuse, modularity, maintainability, security, real-world modelling, extensibility) and cons list (learning curve, run-time overhead, over-design, poor fit for small/numeric tasks).

Q2(b) Architectural modelling: concepts and diagrams.

Architectural modelling describes the physical and high-level organisation of a system: how it is split into deployable pieces and on which hardware those pieces run. In UML it uses components, artifacts, nodes, and packages, shown in component, deployment and package diagrams.

Concepts:

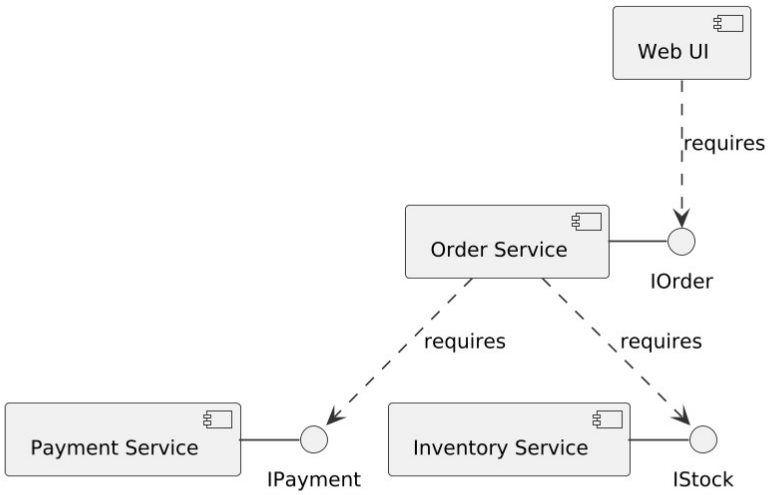
- Component: a replaceable, modular part of a system that encapsulates implementation and exposes interfaces (e.g., `PaymentService`). Drawn as a rectangle with the component icon.
- Interface: a set of operations a component provides (lollipop) or requires (socket).
- Artifact: a physical file produced by development (`.exe`, `.jar`, `.dll`, database script).
- Node: a physical computational resource (server, device, mobile phone), drawn as a 3D box.
- Package: a grouping mechanism for organising model elements (folder icon).
- Collaboration and patterns: groups of elements that work together to provide behaviour; frameworks capture architectural patterns.

Diagrams:

1. Component diagram — shows components, their interfaces and dependencies.
2. Deployment diagram — shows nodes, communication links, and which artifacts run on which node.
3. Package diagram — shows packages and their dependencies (layering).

Example: an online shopping system.

(a) Component diagram



(b) Deployment diagram



Component diagram (a) and deployment diagram (b) of an online shop

Q2(c) Documentation: meaning and considerations in documentation design.

Documentation is the written record of a software system's requirements, design, code, testing and usage. It supports communication between developers, maintainers and users, and is essential for maintenance, training and reuse. In OO development it includes the object, dynamic and functional models, data dictionary, class specifications, user manuals and test reports.

Types: requirements documents; design documents (UML models, architecture); code documentation (comments, API docs); test documents; user documentation (manuals, help).

Considerations in designing documentation:

- Purpose and audience: write for the reader (developer, tester, end user, manager).
- Clarity and simplicity: plain language, defined terms, a data dictionary.
- Completeness: cover all models, interfaces and constraints.
- Consistency: standard templates, naming, notation (UML) across documents.
- Traceability: link requirements → design → code → tests.
- Modularity and organisation: table of contents, indexing, cross-references.

- Up-to-date: document as you go; version control for documents.
- Graphics: use diagrams where structure matters.
- Accessibility: easy to search and navigate (online formats).
- Cost: documentation effort should match project size.

Good documentation reduces maintenance cost and makes it easier for new team members to understand the system.

Q2(d) Namespace, identifiers, variables, constants, enum.

Namespace: a declarative region that groups identifiers to prevent name collisions. Example: `std` namespace holds `cout`, `cin`.

```
namespace Bank { int rate = 7; void show() {} }
int main() { std::cout << Bank::rate; }   // or: using namespace Bank;
```

Identifiers: names given to variables, functions, classes, etc. Rules: letters, digits and _ only; cannot start with a digit; case sensitive; cannot be a keyword. Valid: `total_marks`, `_count`; invalid: `2sum`, `float`, `my-var`.

Variables: named memory locations whose values can change during execution. They must be declared with a type before use: `int age = 20; float pi; char grade = 'A';`.C++ allows declaration anywhere and has local, global, and static variables.

Constants: values that cannot change during execution. Literal constants (`10`, `3.14`, `'A'`, `"Hi"`); `const` qualifier (`const float PI = 3.14159;`); `#define MAX 100` (preprocessor); `constexpr` (compile-time constant).

Enum (enumeration): a user-defined type consisting of named integer constants, improving readability.

```
enum Color { RED, GREEN = 5, BLUE };      // RED=0, GREEN=5, BLUE=6
enum class Day { MON, TUE, WED };         // scoped enum (C++11)
Color c = BLUE;  cout << c;               // 6
```

Q2(e) C++ program depicting multiple inheritance.

In multiple inheritance a derived class inherits from two or more base classes: `class C : public A, public B`. Base constructors run in the order of declaration.

```
#include <iostream>
using namespace std;
class Person {
protected: string name;
public: Person(string n) : name(n) { cout << "Person ctor\n"; }
};
class Employee {
protected: float salary;
public: Employee(float s) : salary(s) { cout << "Employee ctor\n"; }
};
class Manager : public Person, public Employee {   // multiple
```

```
inheritance
    string dept;
public:
    Manager(string n, float s, string d) : Person(n), Employee(s),
dept(d) {}
    void display() {
        cout << "Name: " << name << "\nSalary: " << salary
             << "\nDept: " << dept << endl;
    }
};
int main() {
    Manager m("Anita", 85000, "Sales");
    m.display();
}
```

Output: Person ctor, Employee ctor, then Name: Anita, Salary: 85000, Dept: Sales. If both bases have a member with the same name, resolve with `m.Person::member`.

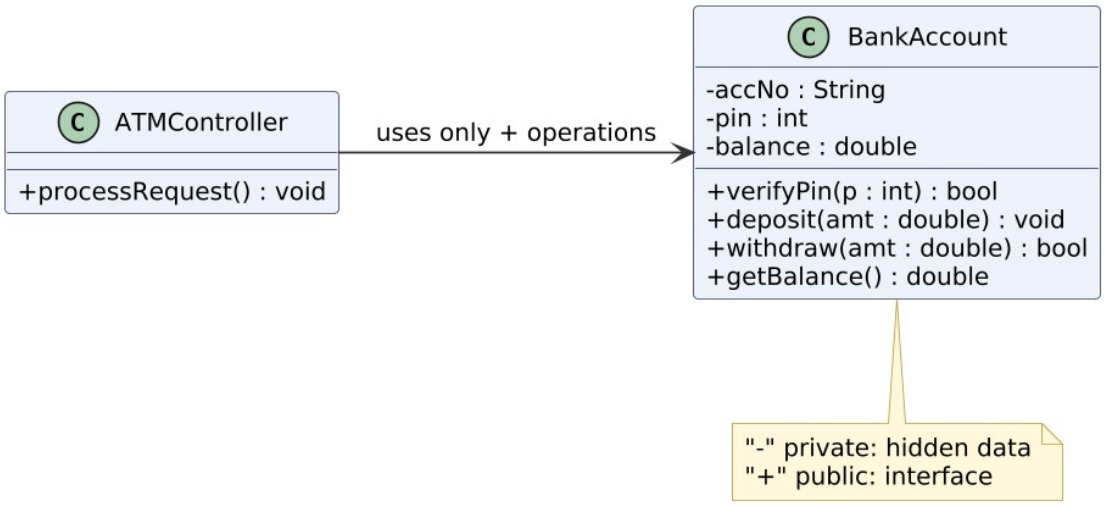
Section C (one part from each question, 5 × 10 = 50)

Q3(a) Concept of encapsulation with example.

Encapsulation is the bundling of data (attributes) and the functions that operate on that data into a single unit, the class, while restricting direct access to the data from outside. Data is kept `private`; access is given through `public` member functions (getters/setters) that can validate inputs. It is the basis of data hiding.

Benefits:

- Data protection: invalid states cannot be set from outside.
- Maintainability: internal representation can change without affecting users.
- Modularity and reuse: a class is a self-contained unit.
- Reduced coupling between parts of a program.



UML class diagram of BankAccount showing encapsulation

```cpp
#include <iostream>
using namespace std;
class BankAccount {
    int accNo;
    double balance;          // cannot be changed directly
public:
    BankAccount(int a) { accNo = a; balance = 0; }
    void deposit(double amt) { if (amt > 0) balance += amt; }
    bool withdraw(double amt) {
        if (amt > 0 && amt <= balance) { balance -= amt; return true;
}
        cout << "Insufficient balance\n"; return false;
    }
    double getBalance() { return balance; }
};
int main() {
    BankAccount acc(1001);
    acc.deposit(5000);
    acc.withdraw(7000);              // rejected by validation
    // acc.balance = 1000000;        // error: balance is private
    cout << acc.getBalance();        // 5000
}
```

Q3(b) Polymorphism with example.

Polymorphism means one name, many forms: the same function call or operator behaves differently depending on the object or arguments. It is of two types: compile-time (function overloading, operator overloading; resolved by the compiler, early binding) and run-time (function overriding through virtual functions; resolved at execution through the vtable, late binding).

Write the classification tree, comparison table and the `Calc/Animal–Dog` program from 2021-22 Q3(a). For a richer run-time example add a `Shape` hierarchy:

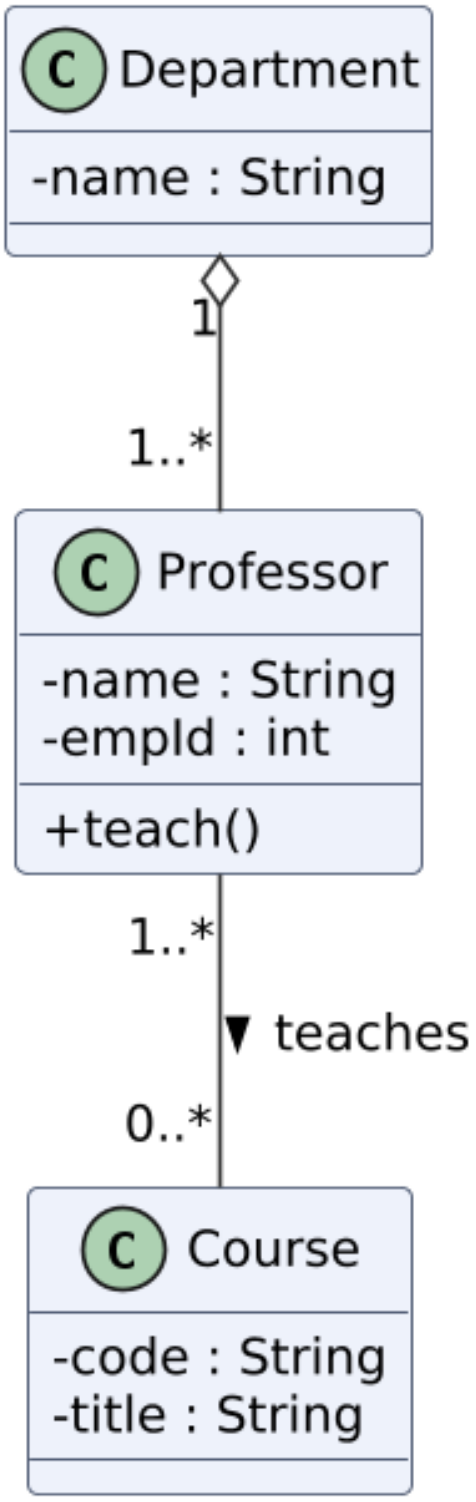
```cpp
class Shape { public: virtual double area() = 0; };
class Circle : public Shape { double r; public: Circle(double x):r(x)
{} double area(){ return 3.14*r*r; } };
class Square : public Shape { double s; public: Square(double x):s(x)
{} double area(){ return s*s; } };
// Shape* arr[2] = { new Circle(1), new Square(2) };
// for (auto p : arr) cout << p->area();   // 3.14  4  — same call,
different behaviour
```

Advantages: extensible code (new subclasses without changing callers), less conditional logic, cleaner interfaces.

Q4(a) Class and object diagrams with examples.

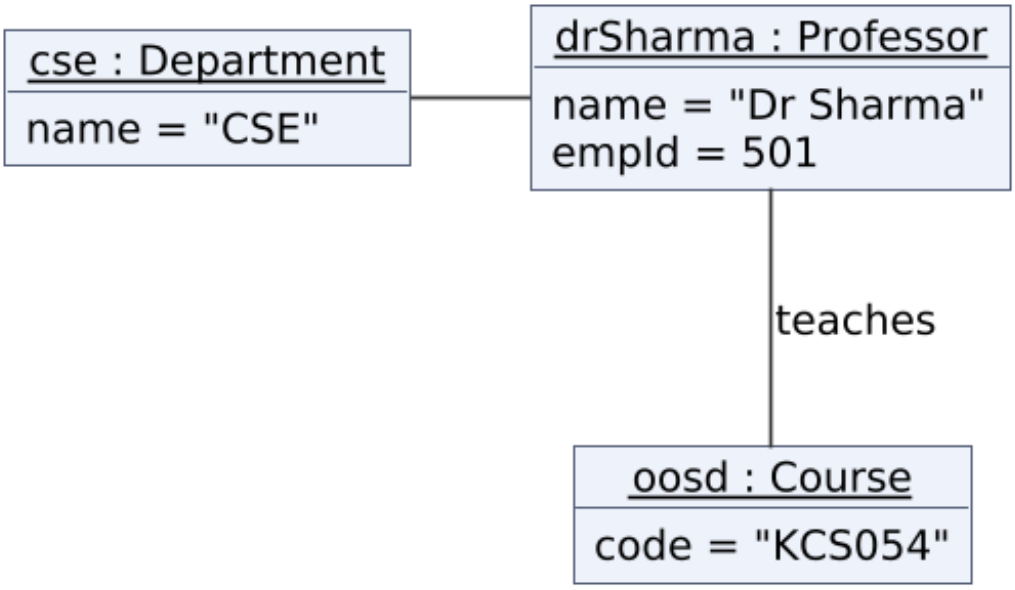
A class diagram is a static structure diagram showing classes, their attributes and operations, and relationships (association, aggregation, composition, generalization, dependency) with multiplicities. It is the main diagram of the object model.

Class notation: a rectangle with three compartments — name, attributes (`visibility name : type`), operations (`visibility name(params) : return`). Visibility: + public, - private, # protected.



Class diagram of a university

An object diagram shows a snapshot of instances and the links between them at a particular moment. Object names are underlined and written `objectName : ClassName`, with attribute values.

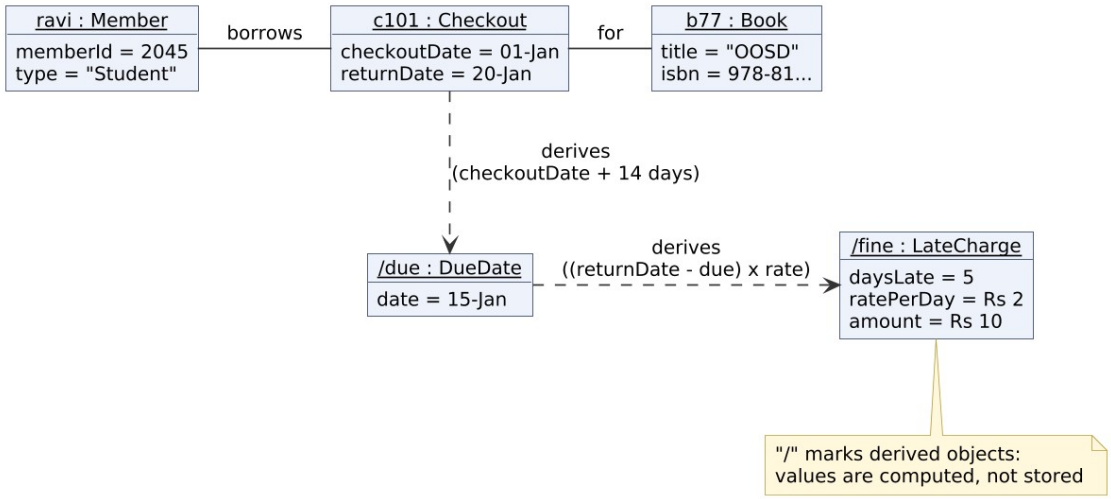


Object diagram (snapshot) of the university

Class diagram	Object diagram
Classes (types)	Objects (instances)
General, valid at all times	Snapshot at one moment
Shows multiplicities	Shows actual links and values

Q4(b) Object diagram for a library book checkout system with due date and late charges as derived objects.

Derived objects (or derived attributes) are computed from other objects; in UML they are marked with a slash `/`. The due date is derived from checkout date + loan period, and the late charge is derived from (return date − due date) × fine per day.



Object diagram for library book checkout with derived due date and late charge

The slash marks objects whose values are calculated, not stored independently. Daily rate and loan period come from the `Member` type (Student: 14 days, Rs 2/day).

Q5(a) Structured analysis and structured design (SA/SD) with example.

SA/SD is a function-oriented methodology. Structured analysis (SA) builds a logical model of what the system does; structured design (SD) converts it into a modular program structure.

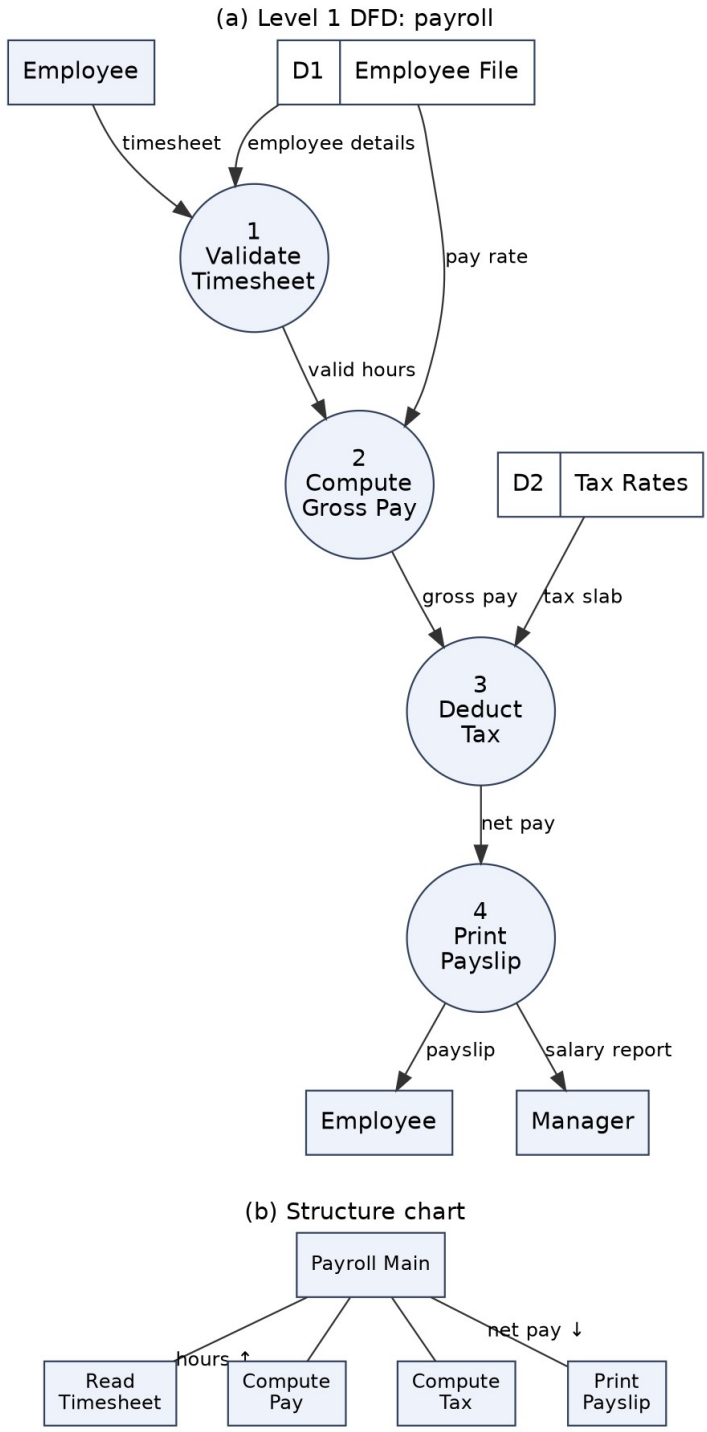
Structured analysis tools:

- Data flow diagrams (DFDs): processes, data flows, data stores, external entities; levelled from context diagram (level 0) down.
- Data dictionary: definitions of all data flows and stores.
- Process specifications: structured English, decision tables/trees.
- State transition diagrams and ER diagrams where needed.

Structured design tools:

- Structure charts: hierarchy of modules and the data/control passed between them.
- Transform analysis and transaction analysis to derive structure charts from DFDs.
- Design goals: high cohesion, low coupling.

Example: payroll system.



Payroll system: Level 1 DFD and structure chart

SA/SD works well for data-processing systems, but changes in function ripple through the design, which motivated object-oriented methods.

Q5(b) Mapping object-oriented concepts using non-object-oriented languages.

An OO design can be implemented in a procedural language such as C by translating each OO concept manually:

OO concept	Implementation in C
Class	`struct` for data + functions taking a pointer to the struct
Object creation	`malloc` + an init function (constructor)
Object destruction	a free/cleanup function (destructor)
Encapsulation	separate `.h` (interface) and `.c` (implementation) files; `static` functions for private
Inheritance	embed base struct as first member of derived struct
Polymorphism / dynamic binding	function pointers in the struct (manual vtable)
Associations	pointers between structs, or arrays/lists of pointers
Method call `obj.f(x)`	`f(&obj, x)`

Steps: translate classes into data structures; pass arguments to methods (explicit self pointer); allocate storage for objects; implement inheritance in data structures; implement method resolution; implement associations; deal with concurrency; encapsulate internal details.

```c
#include <stdio.h>
typedef struct Shape {                  /* base "class" */
    double (*area)(struct Shape*);      /* "virtual" function */
} Shape;
typedef struct { Shape base; double r; } Circle;     /* inheritance */
double circleArea(Shape* s) { Circle* c = (Circle*)s; return 3.14 * c->r * c->r; }
void initCircle(Circle* c, double r) { c->base.area = circleArea; c->r = r; } /* ctor */
int main() {
    Circle c; initCircle(&c, 2);
    Shape* s = (Shape*)&c;
    printf("%.2f\n", s->area(s));      /* polymorphic call: 12.56 */
    return 0;
}
```

Q6(a) Friend function with example.

Same as 2021-22 Q7(a): write the definition, the six characteristics (not in class scope, called without an object, uses `obj.member`, can be declared in any section, friend of several classes, friendship not inherited/mutual/transitive), and the `maxOf(A, B)` program that compares private data of two classes. Add a merit/demerit line: friends give flexibility for operator overloading (`operator<<`) but weaken encapsulation, so use them sparingly.

Q6(b) Virtual function versus pure virtual function, with program.

A virtual function is a member function declared with `virtual` in the base class and redefined in derived classes. When called through a base-class pointer or reference, the derived version runs (run-time polymorphism via the vtable).

Rules for virtual functions: must be members (not static, not friend); accessed through a pointer/reference for dynamic binding; prototype must be the same in base and derived; constructors cannot be virtual but destructors can (and should be, in a polymorphic base).

A pure virtual function is declared by assigning 0: `virtual void draw() = 0;`. It has no definition in the base, making the class abstract; every concrete derived class must override it.

Virtual function	Pure virtual function
Defined in base class	Not defined (`= 0`)
Override optional	Override mandatory in concrete classes
Base can be instantiated	Base becomes abstract, cannot be instantiated
Provides default behaviour	Specifies an interface only

```
#include <iostream>
using namespace std;
class Shape {                           // abstract class
public:
    virtual void info() { cout << "I am a shape\n"; }   // virtual
    virtual double area() = 0;                          // pure
virtual
    virtual ~Shape() {}
};
class Circle : public Shape {
    double r;
public:
    Circle(double x) : r(x) {}
    void info() { cout << "I am a circle\n"; }          // overrides
    double area() { return 3.14 * r * r; }
};
class Rect : public Shape {
    double l, b;
public:
    Rect(double x, double y) : l(x), b(y) {}
    double area() { return l * b; }                      // uses
default info()
};
int main() {
    Shape* s[2] = { new Circle(1), new Rect(2, 3) };
    for (int i = 0; i < 2; i++) {
        s[i]->info();
        cout << "Area = " << s[i]->area() << endl;
        delete s[i];
```

```
    }
    // Shape x;   // error: cannot instantiate abstract class
}
```

Q7(a) Design a class in C++ to create a singly linked list.

```
#include <iostream>
using namespace std;
class Node {
public:
    int data;
    Node* next;
    Node(int d) : data(d), next(nullptr) {}
};
class LinkedList {
    Node* head;
public:
    LinkedList() : head(nullptr) {}
    void insertFront(int d) {
        Node* n = new Node(d);
        n->next = head;
        head = n;
    }
    void insertEnd(int d) {
        Node* n = new Node(d);
        if (!head) { head = n; return; }
        Node* t = head;
        while (t->next) t = t->next;
        t->next = n;
    }
    void remove(int key) {
        Node *t = head, *prev = nullptr;
        while (t && t->data != key) { prev = t; t = t->next; }
        if (!t) { cout << key << " not found\n"; return; }
        if (!prev) head = t->next; else prev->next = t->next;
        delete t;
    }
    bool search(int key) {
        for (Node* t = head; t; t = t->next) if (t->data == key)
return true;
        return false;
    }
    void display() {
        for (Node* t = head; t; t = t->next) cout << t->data << " ->
";
        cout << "NULL\n";
    }
    ~LinkedList() {                    // free all nodes
        while (head) { Node* t = head; head = head->next; delete t; }
    }
};
```

```
int main() {
    LinkedList l;
    l.insertEnd(10); l.insertEnd(20); l.insertFront(5);
l.insertEnd(30);
    l.display();                        // 5 -> 10 -> 20 -> 30 -> NULL
    l.remove(20);
    l.display();                        // 5 -> 10 -> 30 -> NULL
    cout << (l.search(30) ? "Found" : "Not found") << endl;
}
```

Design points: `Node` encapsulates data and link; `LinkedList` hides `head` (private) and exposes operations; the destructor prevents memory leaks.

Q7(b) Define constructor; how it differs from a normal member function.

A constructor is a special member function with the same name as its class, used to initialise objects; it is called automatically when an object is created. Types: default, parameterised, copy, and dynamic constructors.

Constructor	Normal member function
Same name as class	Any valid name
No return type, not even void	Must have a return type
Called automatically at object creation	Called explicitly through an object
Called once per object	Can be called any number of times
Cannot be virtual, static or const	Can be virtual, static, const
Not inherited	Inherited by derived classes
Compiler supplies a default if none written	Never generated by compiler
Used for initialisation	Used for any operation

```
#include <iostream>
using namespace std;
class Student {
    int roll; string name;
public:
    Student(int r, string n) { roll = r; name = n; }   // constructor
    void display() { cout << roll << " " << name << endl; } // member
function
};
int main() {
    Student s(12, "Kiran");   // constructor called automatically
    s.display();              // member function called explicitly
    s.display();              // can be called again
}
```

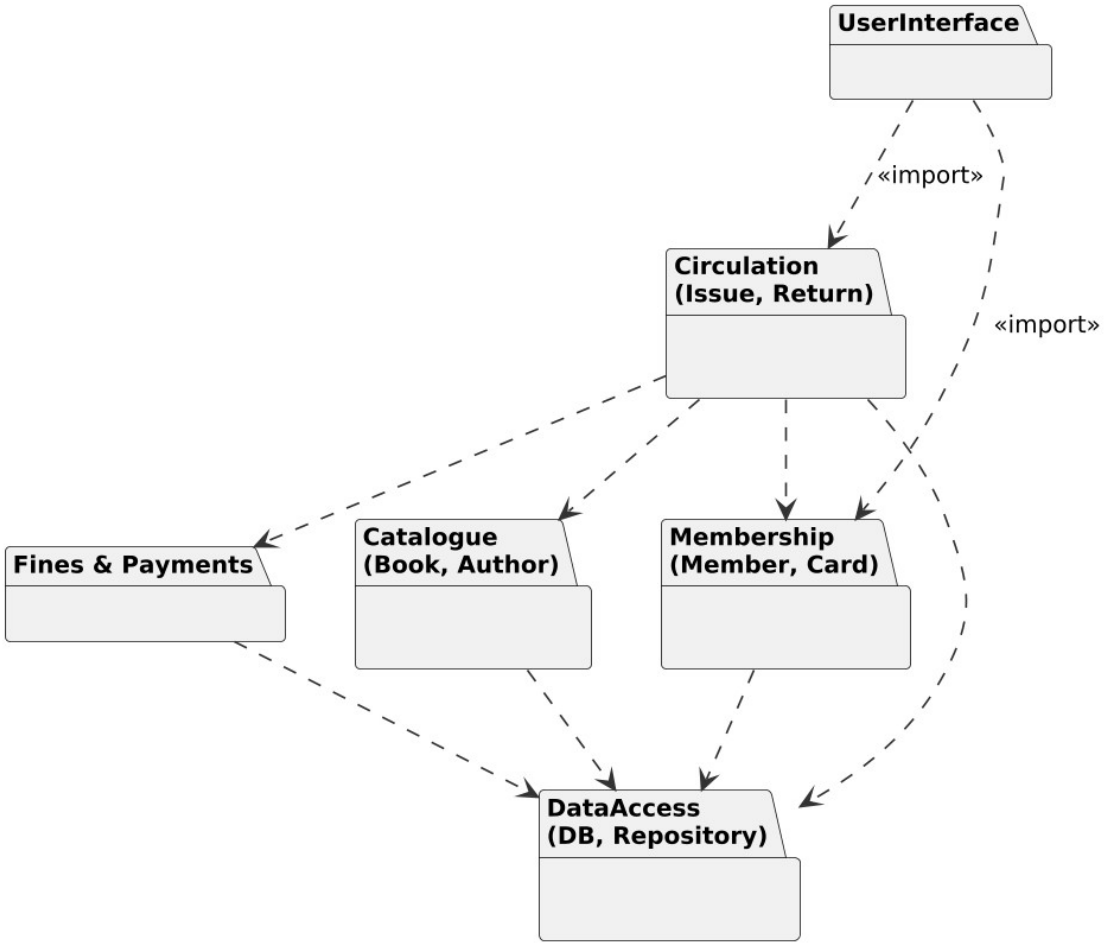
Paper 4: 2024-25 (BCS054, 70 marks)

Section A (2 × 7 = 14)

Q1(a) Significance of modelling in software engineering. A model is a simplified abstraction of a system built before construction. Modelling helps us visualise the system, specify its structure and behaviour, provide templates for construction, and document decisions. It manages complexity by dividing the problem into views, finds design errors early (cheaper to fix), improves communication between users and developers, and supports reuse.

Q1(b) Conceptual model of UML. The conceptual model of UML has three elements: (1) building blocks — things (structural, behavioural, grouping, annotational), relationships (dependency, association, generalization, realization) and diagrams; (2) rules — for names, scope, visibility, integrity and execution that make models well-formed; (3) common mechanisms — specifications, adornments, common divisions (class/object, interface/implementation) and extensibility mechanisms (stereotypes, tagged values, constraints).

Q1(c) Package diagram for a modular library management system.



Package diagram for a modular library management system

Q1(d) How use case diagrams help capture system requirements. Use case diagrams show actors and the goals (use cases) they achieve with the system, defining the system boundary and functional requirements from the user's point of view. They help by: identifying all users and external systems; listing every required function; showing include/extend relations for shared and optional behaviour; giving a common, non-technical language for stakeholders; and serving as the basis for test cases and later design. Example: Customer → Withdraw Cash, Check Balance in an ATM.

Q1(e) Role of inheritance adjustment in design optimization. During design, inheritance is adjusted to increase reuse and simplify classes: (1) rearrange and adjust classes and operations so that similar operations have the same signature; (2) abstract out common behaviour into a new superclass; (3) use delegation instead of inheritance when the subclass is not truly a kind of the superclass (avoid implementation inheritance). This reduces duplicated code and makes the hierarchy cleaner.

Q1(f) Use of enumerations and constants.

```cpp
#include <iostream>
using namespace std;
enum Day { MON = 1, TUE, WED };          // named integer constants
const double PI = 3.14159;               // value cannot change
int main() {
    Day d = TUE;
    cout << d << " " << PI * 2 * 2;      // 2 12.5664
    // PI = 3;  // error: assignment of read-only variable
}
```

Enumerations improve readability by replacing magic numbers with names; constants protect fixed values from accidental change.

Q1(g) Use of the this pointer in C++. `this` is an implicit pointer passed to every non-static member function, pointing to the invoking object. Uses: distinguish members from parameters with the same name (`this->x = x;`), return the current object for chaining (`return *this;`), and pass the current object's address to other functions. Static member functions do not have `this`.

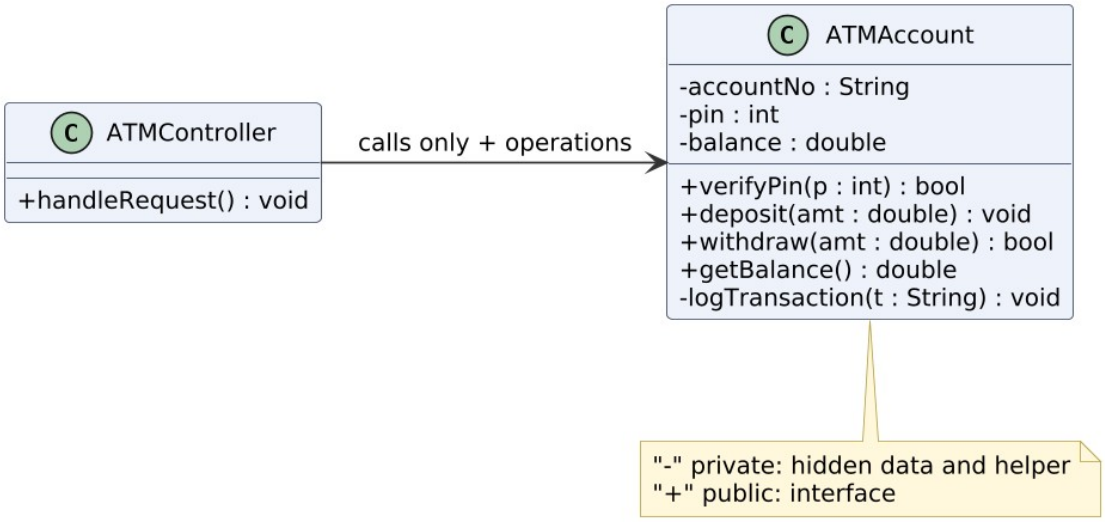
Section B (any 3 × 7 = 21)

Q2(a) Encapsulation and information hiding; how they ensure security and maintainability; UML example.

Encapsulation binds data and the operations on it into one unit (class). Information hiding restricts access to an object's internal details, exposing only a public interface. Encapsulation is the mechanism; information hiding is the principle it enforces.

Security: private data can be changed only through methods that validate input (e.g., a negative deposit is rejected), so objects cannot enter invalid states and sensitive data (PIN, balance) is not directly reachable.

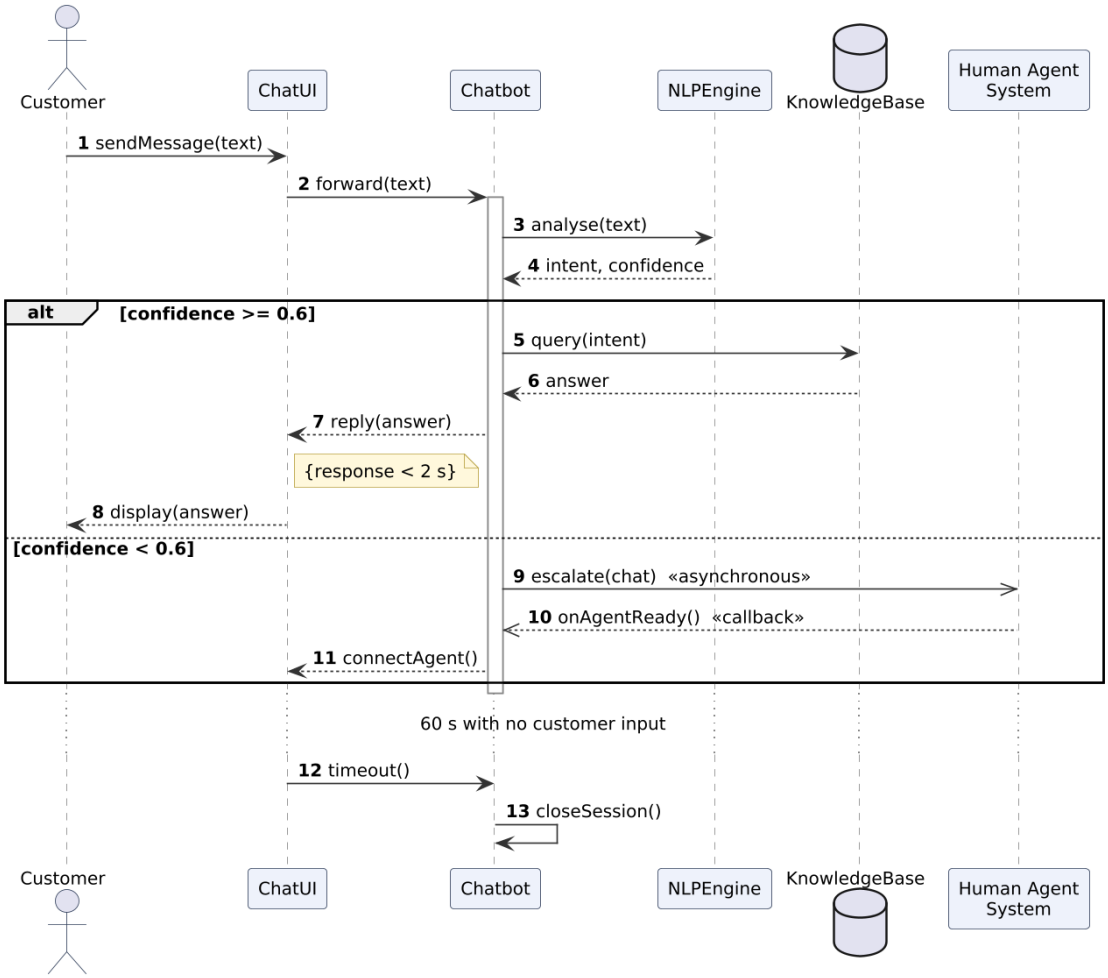
Maintainability: callers depend only on the interface, so the internal representation (e.g., storing balance in paise instead of rupees) can change without touching client code; bugs are localised to one class; classes can be tested independently.



UML class diagram of ATMAccount showing encapsulation and information hiding

Other classes (e.g., `ATMController`) call only the + operations; `pin` and `balance` are never accessed directly.

Q2(b) Interaction (sequence) diagram for a customer service chatbot with time-based interactions.



Sequence diagram of a customer service chatbot

Elements: lifelines (vertical dashed lines), activation bars, synchronous messages (solid arrow), replies (dashed arrow), timing constraints in braces `{response < 2 s}`, a timeout event after 60 s of inactivity, and an `alt` fragment for escalation.

How it aids understanding: it shows the order of messages and which object is responsible for each step; timing constraints document performance requirements; alternative fragments expose error and escalation paths; developers and testers can derive method signatures and test scenarios directly from it.

Q2(c) Process of object-oriented analysis and design for a ride-hailing app.

1. Requirement gathering: interviews and user stories. Riders request rides, see fare estimates, track drivers, pay and rate; drivers accept rides and navigate; admins manage pricing.
2. Use case modelling: actors Rider, Driver, Payment Gateway, Admin; use cases Register, Request Ride, Accept Ride, Track Ride, Make Payment, Rate Trip; `Make Payment` <<include>>d by `Complete Trip`, `Apply Coupon` <<extend>>s `Make Payment`.

3. Domain (object) model: find classes from nouns — `Rider`, `Driver`, `Vehicle`, `Ride`, `Location`, `Fare`, `Payment`, `Rating`. Relationships: Rider 1 — * Ride; Driver 1 — * Ride; Driver 1 — 1 Vehicle; Ride 1 — 1 Payment; `Payment` generalizes `CardPayment`, `UPIPayment`, `CashPayment`.
4. Dynamic model: sequence diagram for Request Ride (Rider → RideService → MatchingService → Driver); state chart for `Ride`: Requested → DriverAssigned → InProgress → Completed → Paid, with Cancelled from Requested/Assigned.
5. Functional model: DFD for fare = base + per-km × distance + per-min × time × surge.
6. System design: architecture as mobile apps + microservices (User, Ride, Matching, Payment, Notification), database, maps API; concurrency for live tracking.
7. Object design: define attributes, operations and algorithms (nearest-driver search), interfaces (`IPaymentMethod`), design patterns (Strategy for pricing, Observer for location updates).
8. Design optimization: cache driver locations in memory, add redundant association Driver → currentRide for fast access, store derived attribute `estimatedFare`, adjust inheritance by abstracting common user data into `User` superclass of `Rider` and `Driver`.
9. Implementation and testing from the models; iterate.

Q2(d) C++ program using a callback mechanism; function pointers and lambdas.

A callback is a function passed as an argument to another function, which calls it back at the right time (e.g., when an event or task completes). In C++ callbacks can be: function pointers (`void (*cb)(int)`), lambda expressions (`[](int x){ ... }`), functors, or `std::function` (which can hold any of these).

```
#include <iostream>
#include <functional>
#include <vector>
using namespace std;

// 1) Callback using a plain function pointer
void onSuccess(int r) { cout << "Function pointer callback: result = "
<< r << endl; }

void compute(int a, int b, void (*callback)(int)) {
    int sum = a + b;
    callback(sum);                    // call back the user's function
}

// 2) Callback using std::function (accepts lambdas too)
class Button {
    vector<function<void()>> handlers;
public:
    void onClick(function<void()> h) { handlers.push_back(h); }
    void click() { for (auto& h : handlers) h(); }   // trigger event
};
```

```
int main() {
    compute(4, 6, onSuccess);                          // function
pointer

    int clicks = 0;
    Button b;
    b.onClick([]() { cout << "Lambda callback: button clicked\n"; });
    b.onClick([&clicks]() { clicks++; });              // lambda
capturing a variable
    b.click();
    b.click();
    cout << "Total clicks = " << clicks << endl;       // 2
}
```

Function pointers work only with free/static functions and carry no state; lambdas can capture local variables, which makes them convenient for event handlers.

Q2(e) Operator overloading for matrix addition and subtraction.

```
#include <iostream>
using namespace std;
class Matrix {
    int r, c;
    int a[10][10];
public:
    Matrix(int rows = 2, int cols = 2) : r(rows), c(cols) {}
    // read elements from user
    void read() {
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++) cin >> a[i][j];
    }
    // overload + : element-wise addition; *this is the left operand
    Matrix operator+(const Matrix& m) const {
        Matrix t(r, c);
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++) t.a[i][j] = a[i][j] + m.a[i]
[j];
        return t;                          // returned by value
    }
    // overload - : element-wise subtraction
    Matrix operator-(const Matrix& m) const {
        Matrix t(r, c);
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++) t.a[i][j] = a[i][j] - m.a[i]
[j];
        return t;
    }
    void show() const {
        for (int i = 0; i < r; i++) {
            for (int j = 0; j < c; j++) cout << a[i][j] << "\t";
            cout << endl;
```

```
        }
    }
};
int main() {
    Matrix A(2, 2), B(2, 2);
    cout << "Enter matrix A (2x2): "; A.read();   // e.g. 5 6 7 8
    cout << "Enter matrix B (2x2): "; B.read();   // e.g. 1 2 3 4
    Matrix S = A + B;   // calls A.operator+(B)
    Matrix D = A - B;   // calls A.operator-(B)
    cout << "A + B:\n"; S.show();                 // 6 8 / 10 12
    cout << "A - B:\n"; D.show();                 // 4 4 / 4 4
}
```

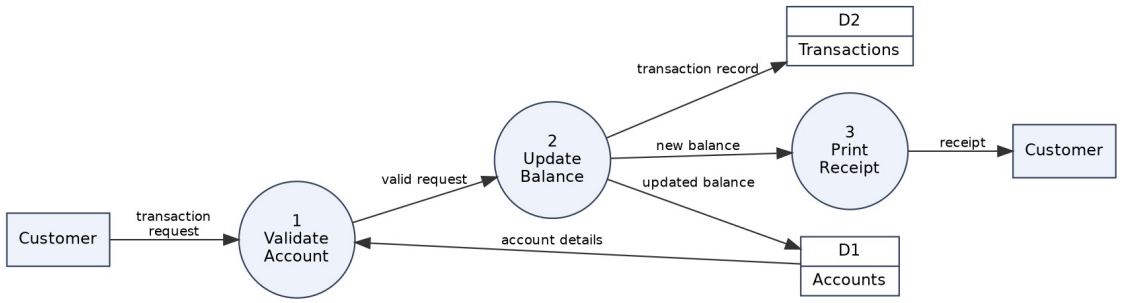
Parameters are passed as `const Matrix&` to avoid copying; functions are `const` because they do not modify the operands. Both matrices must have the same order for these operations.

Section C (one part from each question, 5 × 7 = 35)

Q3(a) Object-oriented modelling versus structured modelling, with a banking system example.

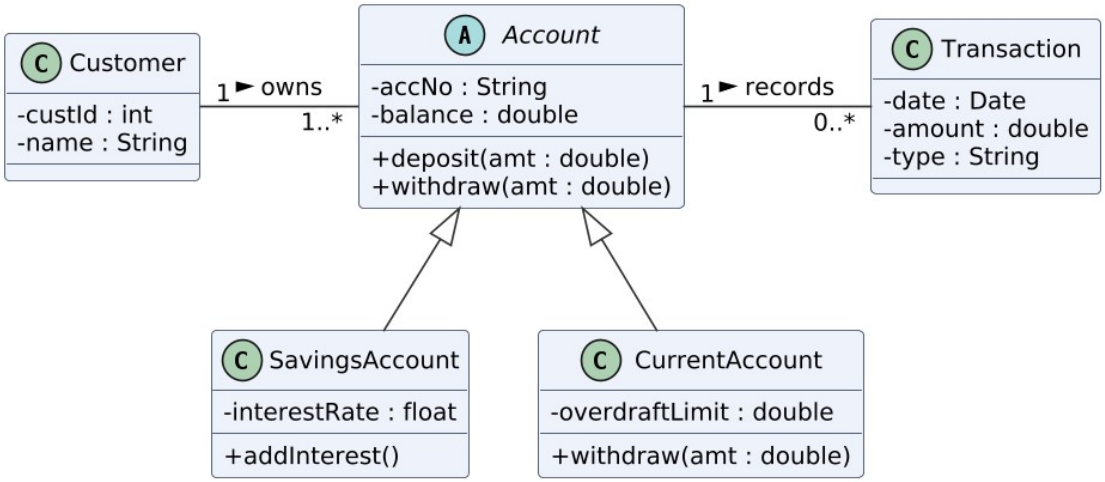
Basis	Structured modelling	Object-oriented modelling
Unit of decomposition	Functions / processes	Objects / classes
Primary model	DFD (functional model)	Class model, then state and interaction models
Data and behaviour	Separate (data stores + processes)	Bound together in classes
Design approach	Top-down	Bottom-up, iterative
Change impact	A change in data ripples across many processes	Change localised to a class
Reuse	Low	High (inheritance, components)
Notation	DFD, data dictionary, structure chart	UML

Banking system, structured view:



DFD of the banking system (structured model)

Banking system, object-oriented view:



Class diagram of the banking system (OO model)

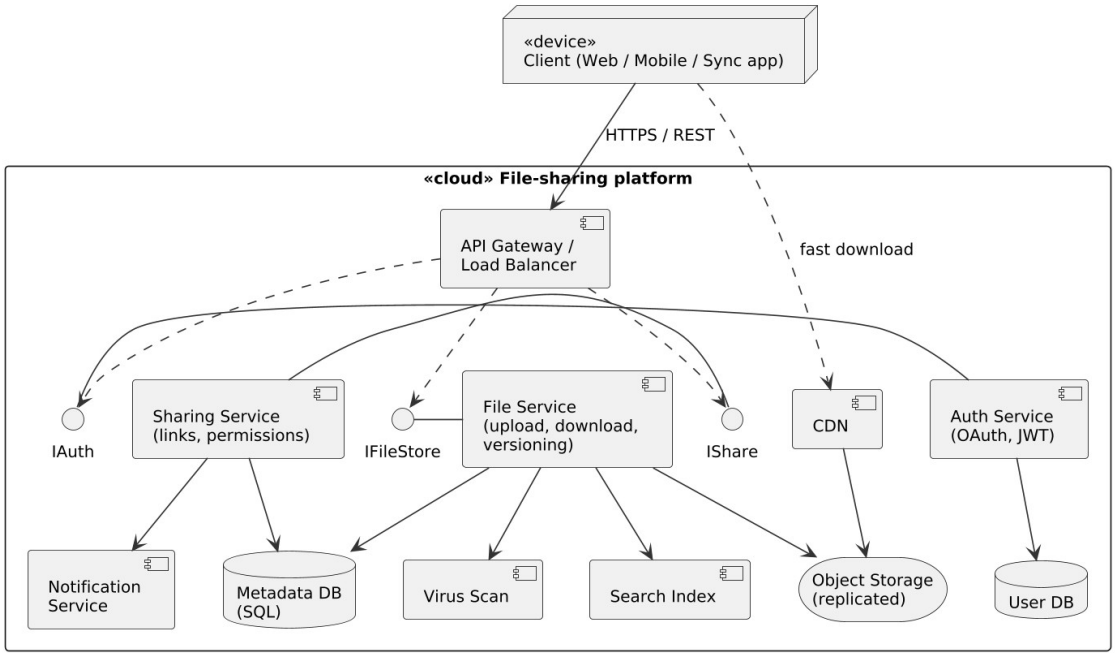
In the structured model, adding a "Fixed Deposit" account type means editing several processes; in the OO model it means adding one subclass that overrides `withdraw()`.

Q3(b) Importance of architecture in OO modelling; UML architecture diagram for a cloud file-sharing system.

Architecture is the high-level structure of a system: its major components, their responsibilities, interfaces and deployment. It matters because it:

- Fixes the system's quality attributes early (scalability, performance, security, availability).
- Divides work among teams through well-defined component interfaces.
- Guides detailed class design and keeps it consistent.
- Enables reuse of components and frameworks.
- Manages complexity and supports evolution (components can be replaced).
- Communicates the big picture to stakeholders.

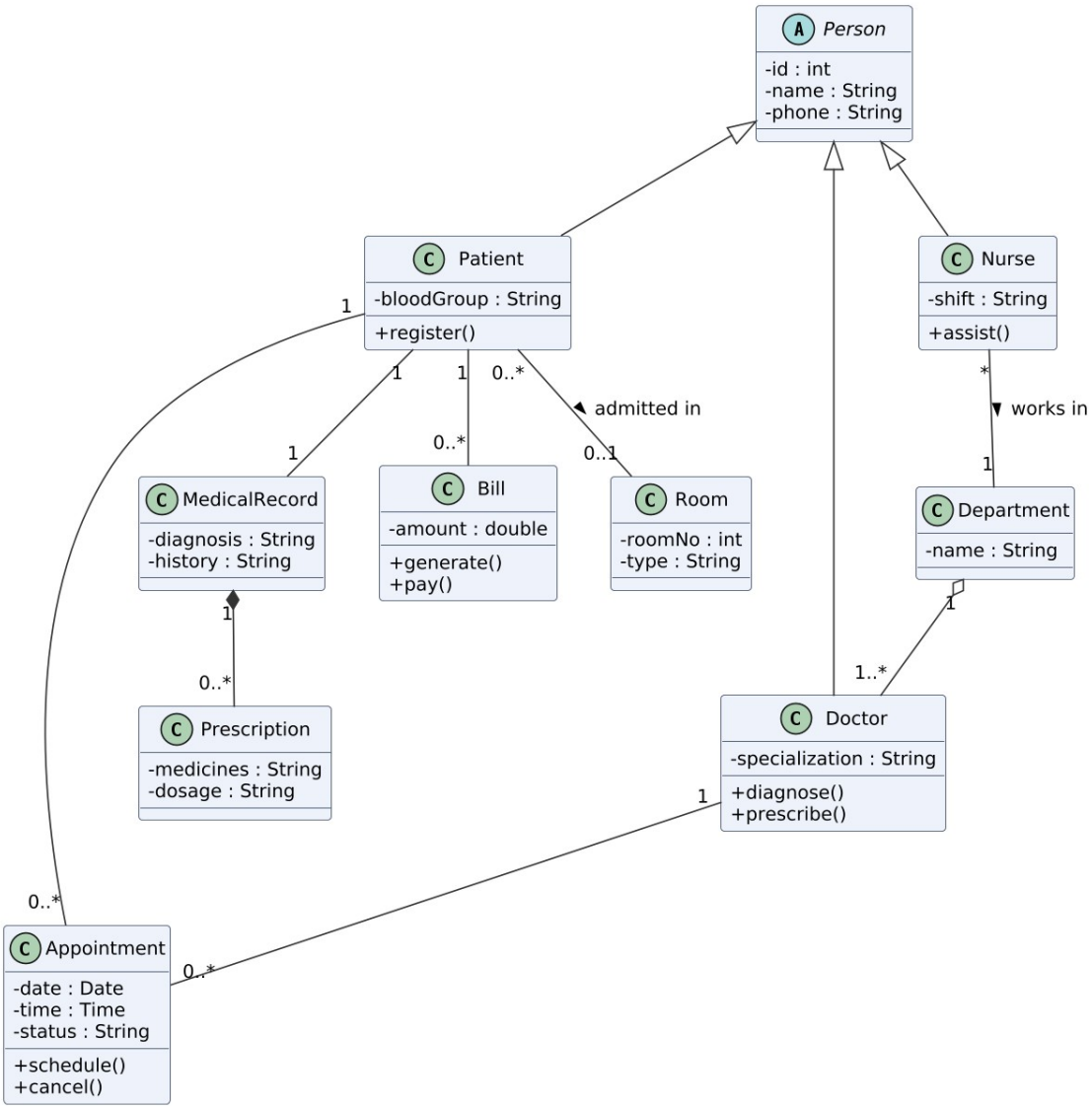
The 4+1 view model (logical, process, development, physical views + use cases) is used to describe it; in UML, component and deployment diagrams show it.



UML architecture of a cloud file-sharing system

Key decisions shown: stateless services behind a load balancer for scalability; file content in replicated object storage, metadata in a database; CDN for fast downloads; separate auth service for security.

Q4(a) UML class diagram for a hospital management system, with design rationale.



UML class diagram of a hospital management system

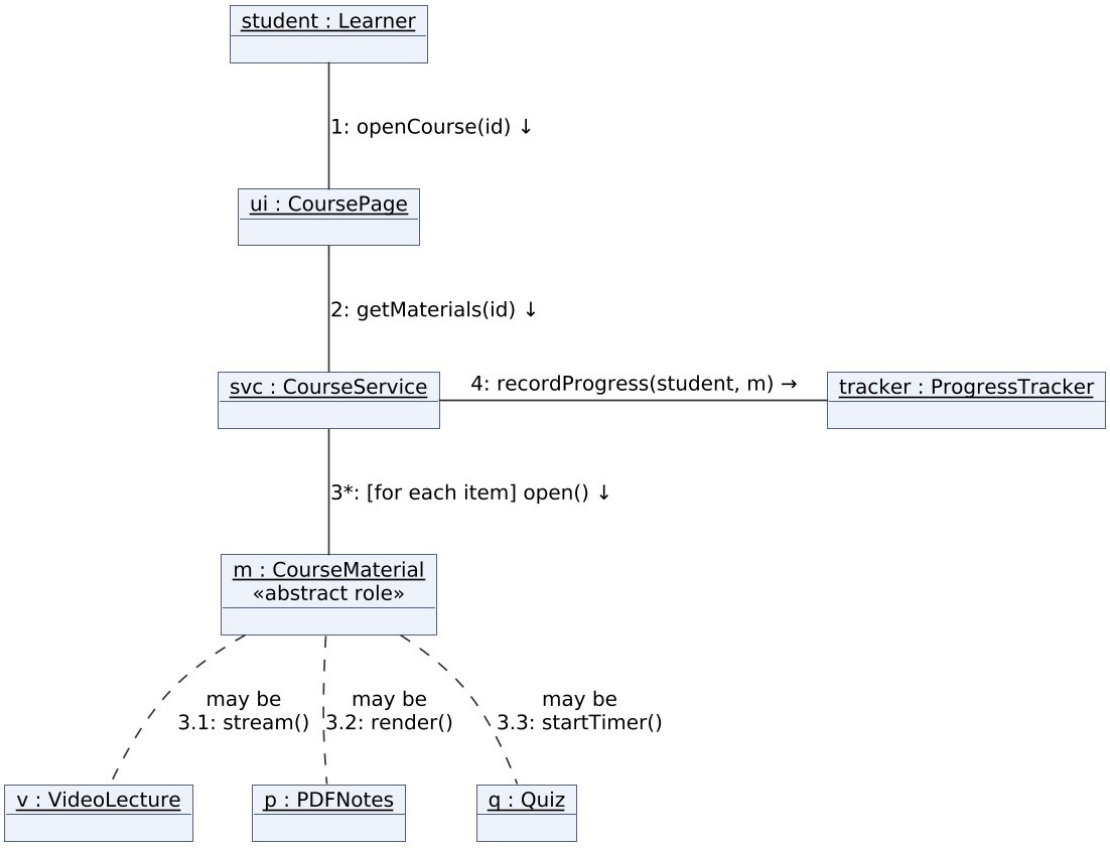
Rationale:

- `Person` is an abstract superclass because patients, doctors and nurses share id, name and contact data (generalization removes duplication).
- `Appointment` is a separate class linking Patient and Doctor (many-to-many over time) and holding its own attributes (date, status).
- `Department` aggregates doctors (hollow diamond): doctors exist even if a department is reorganised.
- `MedicalRecord` composes `Prescriptions` (filled diamond): prescriptions have no meaning without the record.
- Multiplicities encode rules: a patient has one record; a patient may have many bills; a room may be empty.

- Private attributes with public operations enforce encapsulation of sensitive medical data.

Q4(b) Polymorphism in collaboration diagrams; diagram for an online learning platform accessing different course materials.

In a collaboration diagram, polymorphism appears when an object sends the same message to a receiver typed as an abstract class or interface, and the actual receiver at run time may be any subclass, each responding with its own method. The diagram names the receiver by its superclass role (e.g., `m : CourseMaterial`), and a note or separate links show the concrete subclasses that can play that role.



Collaboration diagram of an online learning platform showing polymorphism

The same message `open()` (3*) is sent to every item; a `VideoLecture` streams video, `PDFNotes` renders a document, and a `Quiz` starts a timed test. `CourseService` does not know the concrete type, so new material types (e.g., `Assignment`) can be added without changing it.

Q5(a) C++ program combining the class, state and interaction models in a smart home system.

The class model defines `Device`, `Light`, `Thermostat`, `Hub`; the state model is the `State` enum with legal transitions in `Light::handle()`; the interaction model is the message sequence `User → Hub → Device` (with the hub notifying devices).

```cpp
#include <iostream>
#include <vector>
#include <string>
using namespace std;

// ---------- STATE MODEL ----------
enum class State { OFF, ON, DIMMED };
enum class Event { POWER_ON, POWER_OFF, DIM, MOTION };
string name(State s) { return s == State::OFF ? "OFF" : s == State::ON
? "ON" : "DIMMED"; }

// ---------- CLASS MODEL ----------
class Device {                                    // abstract
superclass
protected:
    string id; State state = State::OFF;
public:
    Device(string i) : id(i) {}
    virtual void handle(Event e) = 0;           // polymorphic
behaviour
    virtual ~Device() {}
};

class Light : public Device {
public:
    Light(string i) : Device(i) {}
    void handle(Event e) override {             // state transitions
        State old = state;
        switch (state) {
            case State::OFF:    if (e == Event::POWER_ON || e ==
Event::MOTION) state = State::ON; break;
            case State::ON:     if (e == Event::DIM) state =
State::DIMMED;
                                else if (e == Event::POWER_OFF) state
= State::OFF; break;
            case State::DIMMED: if (e == Event::POWER_ON) state =
State::ON;
                                else if (e == Event::POWER_OFF) state
= State::OFF; break;
        }
        cout << "  " << id << ": " << name(old) << " -> " <<
name(state) << endl;
    }
};

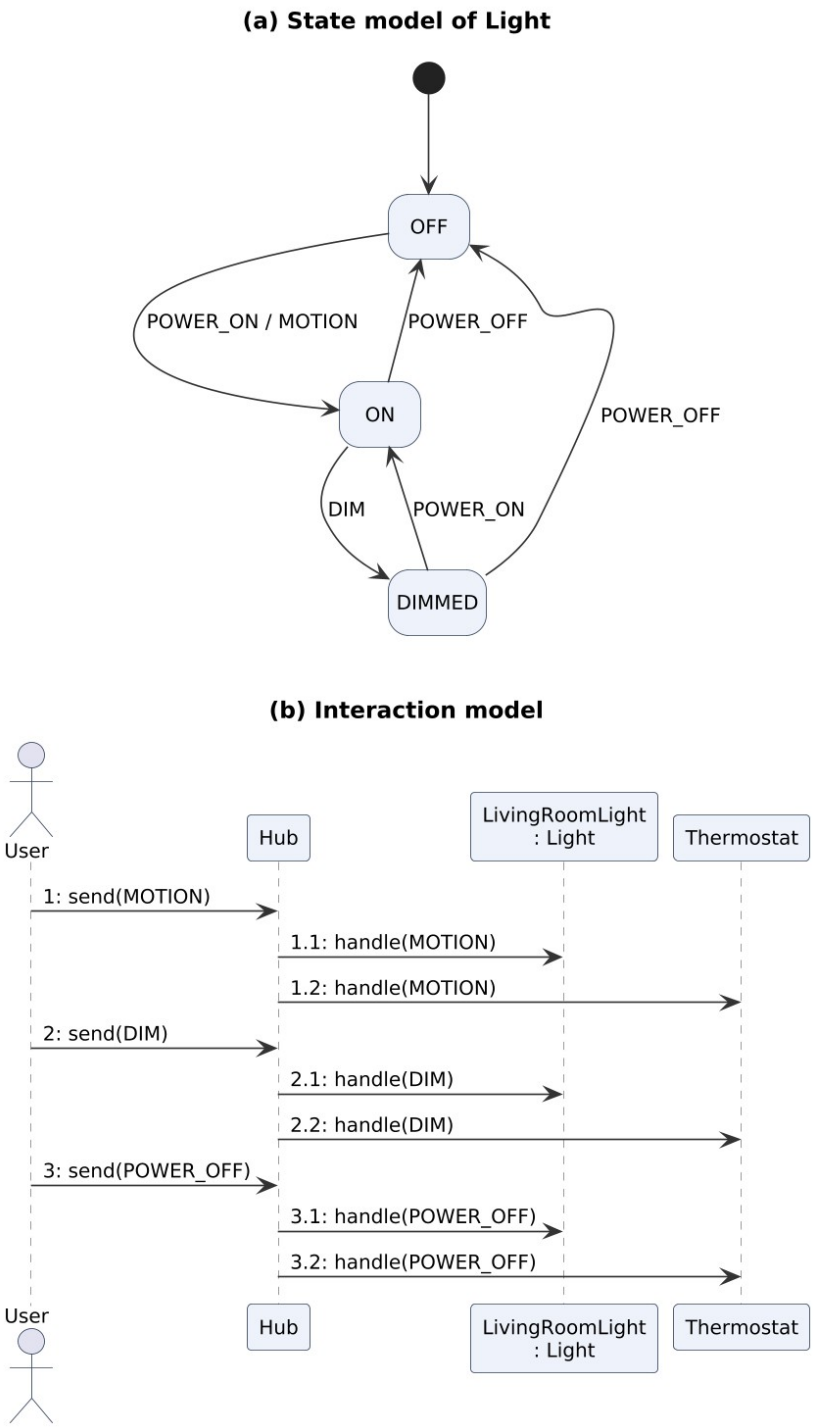
class Thermostat : public Device {
    int temp = 24;
public:
    Thermostat(string i) : Device(i) {}
    void handle(Event e) override {
```

```
        if (e == Event::POWER_ON)  { state = State::ON;  cout << "  "
<< id << ": heating to " << temp << "C\n"; }
        if (e == Event::POWER_OFF) { state = State::OFF; cout << "  "
<< id << ": off\n"; }
    }
};

// ---------- INTERACTION MODEL ----------
class Hub {                                     // mediator: receives
user messages
    vector<Device*> devices;                    // aggregation Hub
<>-- Device
public:
    void add(Device* d) { devices.push_back(d); }
    void send(Event e, string msg) {            // User -> Hub ->
each Device
        cout << "Hub received: " << msg << endl;
        for (Device* d : devices) d->handle(e);
    }
};

int main() {
    Light l1("LivingRoomLight");
    Thermostat t1("Thermostat");
    Hub hub;
    hub.add(&l1); hub.add(&t1);

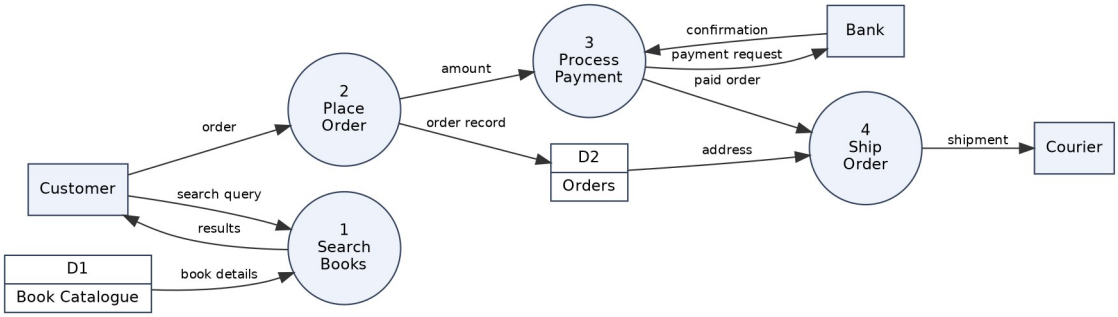
    hub.send(Event::MOTION,    "motion detected");   // Light OFF ->
ON
    hub.send(Event::DIM,       "movie mode");        // Light ON ->
DIMMED
    hub.send(Event::POWER_ON,  "evening scene");     // DIMMED -> ON,
thermostat on
    hub.send(Event::POWER_OFF, "good night");        // all OFF
}
```



Smart home: state model of Light and interaction model

Q5(b) SA/SD versus object-oriented analysis and design: online bookstore case study.

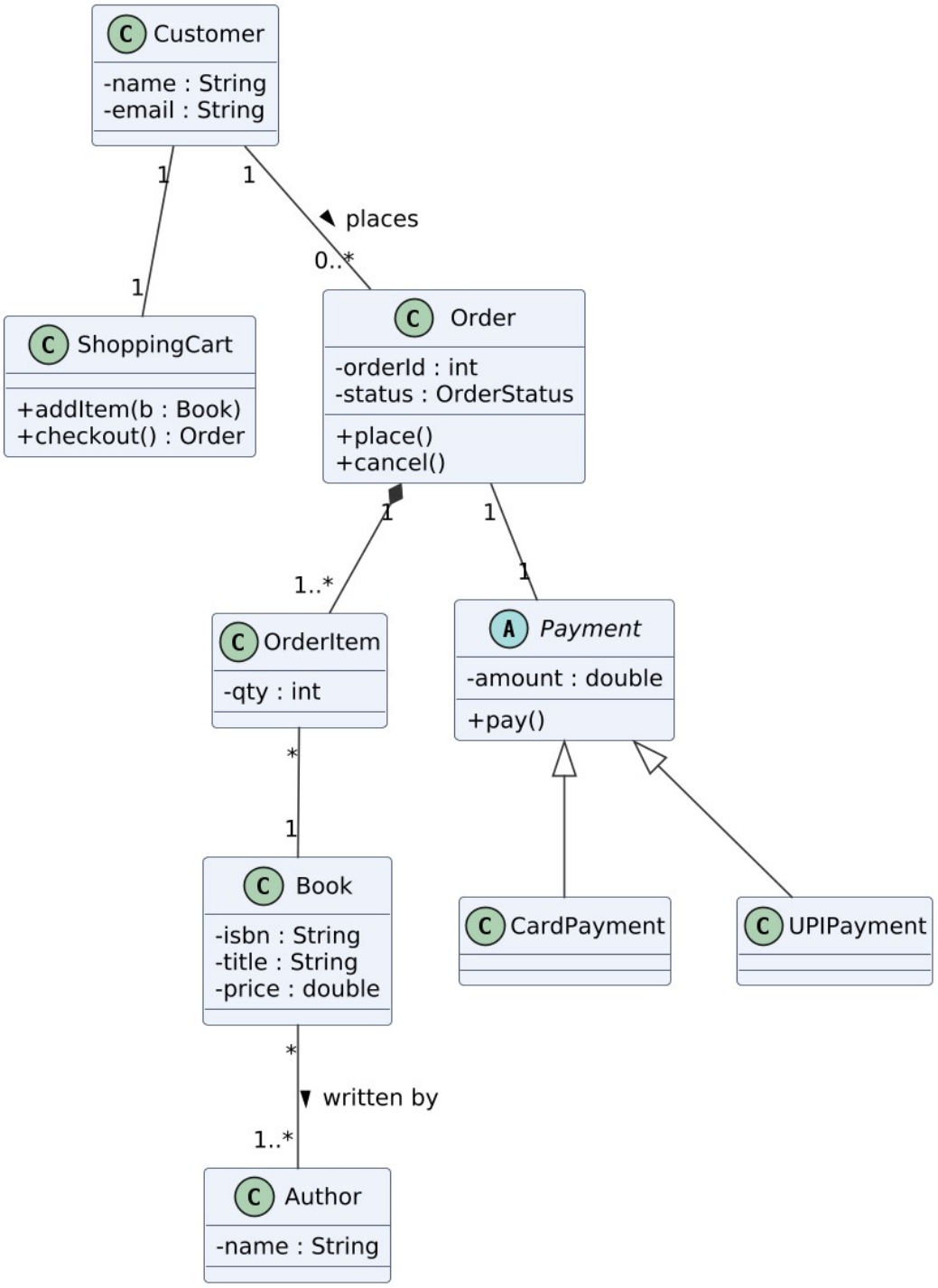
SA/SD view of an online bookstore: the system is decomposed by function.



DFD of the online bookstore (SA/SD view)

Data (catalogue, orders) sits in stores accessed by many processes.

OOAD view: the system is decomposed into collaborating objects.



Class diagram of the online bookstore (OOAD view)

Order states: Placed → Paid → Shipped → Delivered (Cancelled possible from Placed or Paid).

Aspect	SA/SD (bookstore)	OOAD (bookstore)
Decomposition	Processes: Search, Place Order, Pay, Ship	Classes: Book, Customer, Cart, Order, Payment

Aspect	SA/SD (bookstore)	OOAD (bookstore)
Data	Separate stores used by many processes	Encapsulated inside objects
Models	DFD, data dictionary, structure chart	Class, sequence, state, use case diagrams
Adding e-books / a new payment method	Modify several processes and data stores	Add subclass `EBook` or `WalletPayment`
Reuse	Little	`Payment`, `Cart` reusable in other shops
Maps to code	Functions in C	Classes in C++/Java
Best suited to	Stable, data-processing systems	Evolving, interactive systems

Q6(a) Inline functions versus macros: program and performance comparison.

```
#include <iostream>
using namespace std;
#define SQUARE_MACRO(x) x * x            // textual substitution, no
type check
inline int squareInline(int x) { return x * x; }   // compiler-checked

int main() {
    int a = 5;
    cout << "Inline: " << squareInline(a + 1) << endl;   // 36
(correct)
    cout << "Macro : " << SQUARE_MACRO(a + 1) << endl;   // 5 + 1*5 +
1 = 11 (wrong)
    int b = 3;
    cout << "Inline: " << squareInline(b++) << endl;     // 9, b
incremented once
    // SQUARE_MACRO(b++) would expand to b++ * b++  (undefined
behaviour)
}
```

Basis	Inline function	Macro
Processed by	Compiler	Preprocessor (before compilation)
Type checking	Yes	No
Argument evaluation	Once	Possibly multiple times (side effects)
Execution speed	Fast: call overhead removed, like a macro	Fast: no call overhead
Memory (code size)	Increases code size at each call site if expanded; compiler may refuse for large	Always expanded at every use, increasing code size

Basis	Inline function	Macro
	functions	
Debugging	Can be debugged; has scope	Hard to debug; no scope
Class member access	Can access private members	Cannot

Conclusion: both remove call overhead, so speed is similar; inline functions are safer and the compiler can choose not to inline large functions to control code size, so they are preferred in C++.

Q6(b) Function overloading to calculate volume of cube, cylinder and sphere.

```
#include <iostream>
using namespace std;
const double PI = 3.14159;

// Function overloading: same name 'volume', different parameter
lists.
// The compiler selects the version by number/type of arguments
(compile-time polymorphism).

// Cube: one int argument (side)
double volume(int side) {
    return side * side * side;              // a^3
}
// Cylinder: two double arguments (radius, height)
double volume(double r, double h) {
    return PI * r * r * h;                  // pi r^2 h
}
// Sphere: one double argument (radius) -> differs from cube by type
double volume(double r) {
    return (4.0 / 3.0) * PI * r * r * r;    // 4/3 pi r^3
}

int main() {
    cout << "Cube (a=3):             " << volume(3) << endl;
// 27
    cout << "Cylinder (r=2, h=5):    " << volume(2.0, 5.0) << endl;
// 62.8318
    cout << "Sphere (r=1.5):         " << volume(1.5) << endl;
// 14.1372
}
```

Commentary: `volume(3)` matches `int` exactly, so the cube version runs; `volume(1.5)` matches `double` and calls the sphere version; two arguments select the cylinder. The return type alone cannot distinguish overloads; parameters must differ.

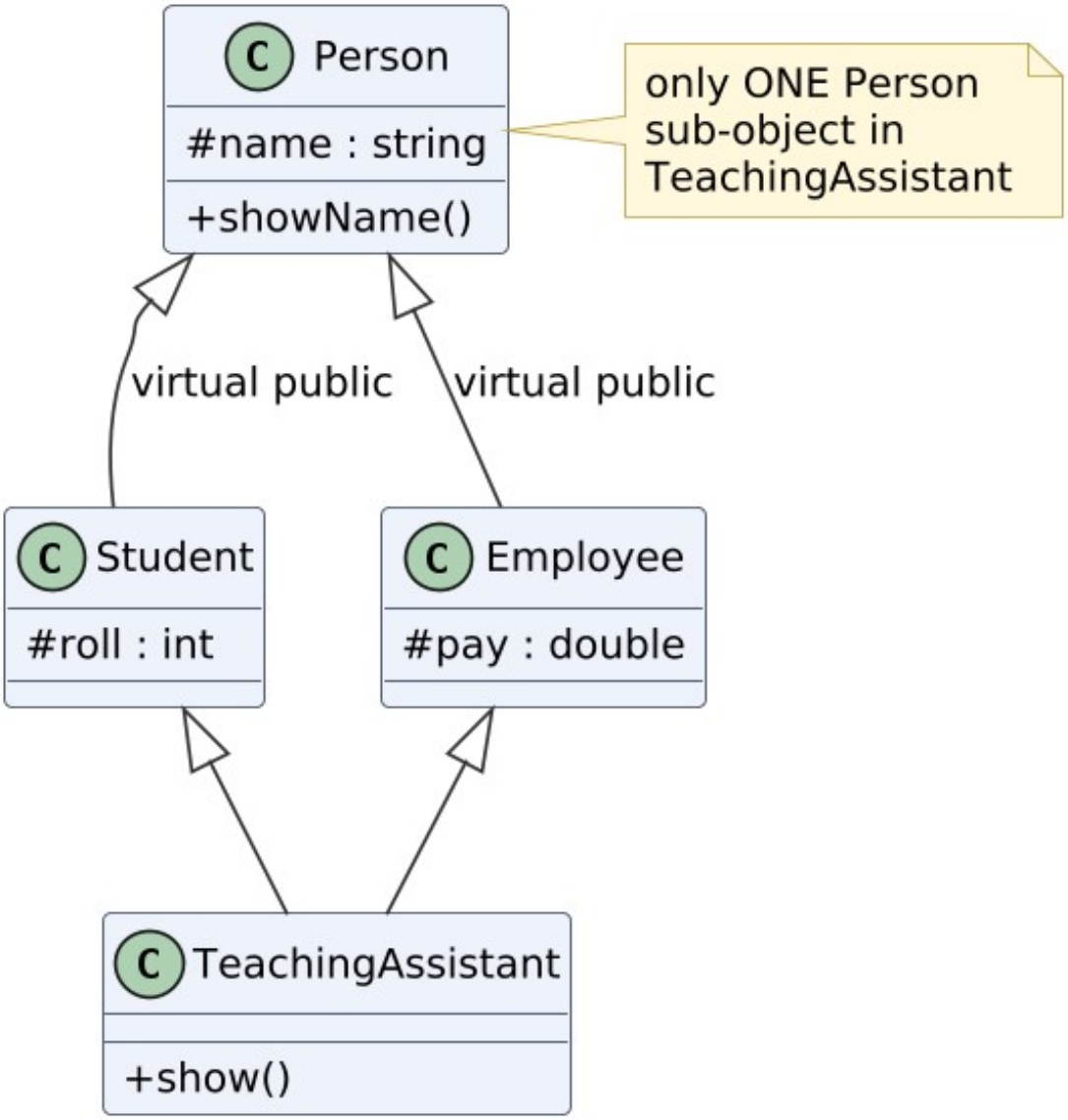
Q7(a) Static data members and functions; program to track the number of objects.

A static data member is declared with `static` inside a class; one copy is shared by all objects, it exists even before any object is created, is zero-initialised by default, and must be defined outside the class. A static member function belongs to the class, can access only static members, has no `this` pointer, and is called with `ClassName::f()`.

```
#include <iostream>
using namespace std;
class Counter {
    static int alive;      // objects currently existing
    static int created;    // total ever created
public:
    Counter()  { created++; alive++; }
    ~Counter() { alive--; }
    static void report() {
        cout << "Created: " << created << ", Alive: " << alive <<
endl;
    }
};
int Counter::alive = 0;
int Counter::created = 0;
int main() {
    Counter::report();             // Created: 0, Alive: 0
    Counter a, b;
    {
        Counter c;
        Counter::report();         // Created: 3, Alive: 3
    }                              // c destroyed
    Counter::report();             // Created: 3, Alive: 2
}
```

Q7(b) Hybrid inheritance and resolving ambiguity with virtual base classes.

Hybrid inheritance combines two or more types of inheritance. The common diamond form combines hierarchical (A → B, A → C) and multiple (B, C → D) inheritance. Without a virtual base, D contains two copies of A, so accessing A's member from D is ambiguous (compiler error). Declaring `virtual public A` in B and C makes them share one copy of A; the most-derived class D then constructs A directly.



Hybrid (diamond) inheritance with virtual base class

```
#include <iostream>
using namespace std;
class Person {
protected: string name;
public:
    Person(string n = "") : name(n) { cout << "Person constructed\n";
}
    void showName() { cout << "Name: " << name << endl; }
};
class Student : virtual public Person {           // virtual base
protected: int roll;
public: Student(int r) : roll(r) {}
};
class Employee : virtual public Person {          // virtual base
```

```
protected: double stipend;
public: Employee(double s) : stipend(s) {}
};
class TeachingAssistant : public Student, public Employee {   //
hybrid
public:
    TeachingAssistant(string n, int r, double s)
        : Person(n), Student(r), Employee(s) {}  // D calls virtual
base ctor
    void show() {
        showName();                                  // no ambiguity
        cout << "Roll: " << roll << ", Stipend: " << stipend << endl;
    }
};
int main() {
    TeachingAssistant ta("Meera", 42, 15000);
    ta.show();
}
```

Output: “Person constructed” is printed only once, proving a single shared `Person`. Without `virtual`, `ta.showName()` would give an “ambiguous” compile error and would need `ta.Student::showName()` to resolve it manually.